Introduction to Basic and Advanced Statistical Modelling

From LM to GLM

Alexandre Courtiol

Table of contents

Linear Models (LM)

LM — what is a statistical model?

Statistical model

an approximate description of the *data generating process*, i.e. an approximation of how the world generates data

LM — what is a statistical model?

Statistical model

an approximate description of the *data generating process*, i.e. an approximation of how the world generates data

NB: seeing models as a *data generating process* may not be intuitive at first, but it is both conceptually and practically crucial!

In particular, it helps

- correctly interpreting model parameters
- planning sample size before data collection
- testing the coverage of confidence intervals (i.e. do they work as intended)
- testing the robustness of the statistical test (i.e. check the rate of false positives)
- performing prior and posterior predictive checks during Bayesian analyses

LM — what is a linear model (LM)?

Linear model

a data generating process produced by a *linear function*: the data are generated from a set of terms by multiplying each term by a constant (a model parameter) and summing them

LM — what is a linear model (LM)?

Linear model

a data generating process produced by a *linear function*: the data are generated from a set of terms by multiplying each term by a constant (a model parameter) and summing them

NB:

- linear models encompass the classic linear regression which forms a straight line, but also many other models (e.g. polynomials are linear and yet do not form straight lines) → LM are linear in parameters (not necessarily in variables)
- much of the difficulty people encounter when dealing with more complex models (e.g. Generalised Linear Models, Linear Mixed-effects Models, Hierarchical Models) is often caused by not having sufficiently understood the LM

LM — notations of a LM: simple notation

$$y_i = \beta_0 + \beta_1 \times x_{1,i} + \beta_2 \times x_{2,i} + \cdots + \beta_p \times x_{p,i} + \epsilon_i \text{ with } \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

LM — notations of a LM: simple notation

$$y_i = \beta_0 + \beta_1 \times x_{1,i} + \beta_2 \times x_{2,i} + \cdots + \beta_p \times x_{p,i} + \epsilon_i \text{ with } \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

- y_i = the response variable / dependent variable / observations to explain
- β_j = the model parameters / regression coefficients
- $x_{j,i}$ = regressors / constants derived from the predictor variables (aka predictors for short)
- ϵ_i = the errors / residual errors

LM — notations of a LM: simple notation

$$y_i = \beta_0 + \beta_1 \times x_{1,i} + \beta_2 \times x_{2,i} + \cdots + \beta_p \times x_{p,i} + \epsilon_i \text{ with } \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

- y_i = the response variable / dependent variable / observations to explain
- β_j = the model parameters / regression coefficients
- $x_{j,i}$ = regressors / constants derived from the predictor variables (aka predictors for short)
- ϵ_i = the errors / residual errors

NB:

- this is the notation that biologists should use in their paper/thesis to describe their model
- the regressors are sometimes directly given by the columns from your dataset (i.e. the predictors), but not always (more on that later)
- when parameters are not known but estimated they should wear a hat (e.g. $\widehat{\beta_0}$) and ϵ_i (also sometimes called ε_i) usually becomes e_i or r_i

LM — notations of a LM: usual R notation

```
y ~ x_1 + x_2 + ... + x_p
```

NB:

- the parameters and the error term are not written down

LM — notations of a LM: matrix notation

$$Y = X\beta + \epsilon$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ ... \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & ... & x_{1,p} \\ 1 & x_{2,1} & x_{2,2} & ... & x_{2,p} \\ 1 & x_{3,1} & x_{3,2} & ... & x_{3,p} \\ ... & ... & ... & ... & ... \\ 1 & x_{n,1} & x_{n,2} & ... & x_{n,p} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ ... \\ \beta_p \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \\ ... \\ \epsilon_n \end{bmatrix}$$

LM — notations of a LM: matrix notation

$$Y = X\beta + \epsilon$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ ... \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & ... & x_{1,p} \\ 1 & x_{2,1} & x_{2,2} & ... & x_{2,p} \\ 1 & x_{3,1} & x_{3,2} & ... & x_{3,p} \\ ... & ... & ... & ... & ... \\ 1 & x_{n,1} & x_{n,2} & ... & x_{n,p} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ ... \\ \beta_p \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \\ ... \\ \epsilon_n \end{bmatrix}$$

- $Y =$ the vector of observations
- $X =$ a matrix called the design (or model) matrix
- $\beta =$ the vector of model parameters
- $\epsilon =$ the vector of errors, with again $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$

NB: this is the notation for statisticians and computer scientists, which is particularly useful when interpreting coefficients

LM — data generating process: data simulation

Let's enter **God mode** → we pretend that Emerald Lion (a goddess) knows how her world generates data; she informed us that in her world aliens keep growing with the number of humans they eat as follows:

$$\begin{cases} \text{size}_i = 50 + 1.5 \times \text{humans\_eaten}_i + \epsilon_i \\ \epsilon_i \sim \mathcal{N}(0, \sigma^2 = 4) \end{cases}$$

LM — data generating process: data simulation

Let's enter **God mode** → we pretend that Emerald Lion (a goddess) knows how her world generates data; she informed us that in her world aliens keep growing with the number of humans they eat as follows:

$$\begin{cases} \text{size}_i = 50 + 1.5 \times \text{humans\_eaten}_i + \epsilon_i \\ \epsilon_i \sim \mathcal{N}(0, \sigma^2 = 4) \end{cases}$$

We can thus generate the size of 12 aliens that have eaten 1–3 humans following E.L.'s recipe:

```
set.seed(2)
N <- 12
Alien <- tibble(humans_eaten = rep(1:3, each = 4), # alternative e.g. rpois() would do
                intercept = 50, slope = 1.5,
                error = rnorm(n = N, mean = 0, sd = 2), # must be rnorm!
                size = intercept + slope*humans_eaten + error)
```

LM — data generating process: data simulation

Let's enter **God mode** → we pretend that Emerald Lion (a goddess) knows how her world generates data; she informed us that in her world aliens keep growing with the number of humans they eat as follows:

$$\begin{cases} \text{size}_i = 50 + 1.5 \times \text{humans\_eaten}_i + \epsilon_i \\ \epsilon_i \sim \mathcal{N}(0, \sigma^2 = 4) \end{cases}$$

We can thus generate the size of 12 aliens that have eaten 1–3 humans following E.L.'s recipe:

```
set.seed(2)
N <- 12
Alien <- tibble(humans_eaten = rep(1:3, each = 4), # alternative e.g. rpois() would do
                intercept = 50, slope = 1.5,
                error = rnorm(n = N, mean = 0, sd = 2), # must be rnorm!
                size = intercept + slope*humans_eaten + error)
```

NB: we could also write the last 2 lines as:

```
size <- rnorm(n = 12, mean = intercept + slope*humans_eaten, sd = 2)
```

LM — data generating process: data check

Simulated data

```
Alien
```

```
# A tibble: 12 x 5
  humans_eaten intercept slope  error  size
         <int>     <dbl> <dbl>  <dbl> <dbl>
1            1        50   1.5 -1.79   49.7
2            1        50   1.5  0.370  51.9
3            1        50   1.5  3.18   54.7
4            1        50   1.5 -2.26   49.2
5            2        50   1.5 -0.161  52.8
# i 7 more rows
```

- which columns show the *model parameters*?
- which columns show *regressor* (here = predictor)?
- which columns show *errors*?
- which columns show *response variable*?

Practice 1

Simulated data

```
Alien
```

```
# A tibble: 12 x 5
  humans_eaten intercept slope  error  size
         <int>     <dbl> <dbl>  <dbl> <dbl>
1            1        50   1.5 -1.79   49.7
2            1        50   1.5  0.370  51.9
3            1        50   1.5  3.18   54.7
4            1        50   1.5 -2.26   49.2
5            2        50   1.5 -0.161  52.8
# i 7 more rows
```

- which columns show the *model parameters*?
- which columns show *regressor* (here = predictor)?
- which columns show *errors*?
- which columns show *response variable*?
- what would the *mean response value* be, for an alien who does not eat humans?
- what would the *mean response value* be, for an alien who eats 2 humans?

Solution

- what would the *mean response value* be, for an alien who does not eat humans?

$$\text{mean}(y \mid \text{humans\_eaten} = 0) = 50 + 1.5 \times 0 = 50$$

Solution

- what would the *mean response value* be, for an alien who does not eat humans?

$$\text{mean}(y \mid \text{humans\_eaten} = 0) = 50 + 1.5 \times 0 = 50$$

- what would the *mean response value* be, for an alien who eats 2 humans?

$$\text{mean}(y \mid \text{humans\_eaten} = 2) = 50 + 1.5 \times 2 = 53$$

LM — model fitting: frequentist approach

In practice, model parameters are unknown, so we have to estimate them
$\rightarrow$ that is the whole point of statistical modelling... $\rightarrow$ the God mode is now off

LM — model fitting: frequentist approach

In practice, model parameters are unknown, so we have to estimate them
→ that is the whole point of statistical modelling... → the God mode is now off

- in frequentist statistics, *fitting the model* means estimating the values of the model parameters that maximise the probability of the data

```
fit_alien_lm <- lm(size ~ humans_eaten, data = Alien)
summary(fit_alien_lm) # or coef(fit_alien_lm) for a focus on coefficients only
```

```
Call:
lm(formula = size ~ humans_eaten, data = Alien)

Residuals:
    Min      1Q  Median      3Q     Max
-1.9710 -1.1744 -0.4725  0.7023  3.4655

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   48.8353     1.3454  36.298 5.99e-12 ***
humans_eaten   2.3749     0.6228   3.813  0.00341 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.762 on 10 degrees of freedom
Multiple R-squared:  0.5925,    Adjusted R-squared:  0.5518
F-statistic: 14.54 on 1 and 10 DF,  p-value: 0.003411
```

LM — model fitting: interlude on ordinary least squares (OLS)

In LM (but not GLM), finding the estimates that maximise the probability of the data given the model (i.e. maximising the likelihood) is equivalent to minimising the sum of the squared residuals, so this is what `lm()` actually does since it is computationally simpler than computing the likelihood

To best illustrate the OLS concept, we need to simulate data a little more spread out

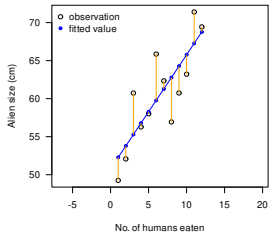
```
set.seed(123)
Alien2 <- tibble(humans_eaten = 1:12, intercept = 50, slope = 1.5,
                 size = rnorm(n = 12, mean = intercept + slope*humans_eaten, sd = 4))
fit_alien2_lm <- lm(size ~ humans_eaten, data = Alien2)
```

LM — model fitting: interlude on ordinary least squares (OLS)

In LM (but not GLM), finding the estimates that maximise the probability of the data given the model (i.e. maximising the likelihood) is equivalent to minimising the sum of the squared residuals, so this is what `lm()` actually does since it is computationally simpler than computing the likelihood

To best illustrate the OLS concept, we need to simulate data a little more spread out

```
set.seed(123)
Alien2 <- tibble(humans_eaten = 1:12, intercept = 50, slope = 1.5,
                 size = rnorm(n = 12, mean = intercept + slope*humans_eaten, sd = 4))
fit_alien2_lm <- lm(size ~ humans_eaten, data = Alien2)
```



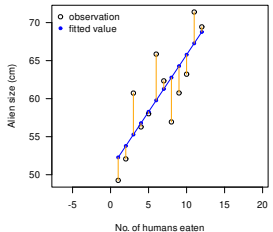
Residuals on best fit

LM — model fitting: interlude on ordinary least squares (OLS)

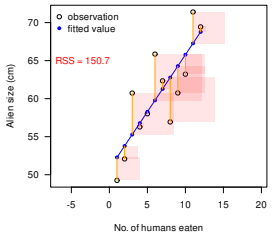
In LM (but not GLM), finding the estimates that maximise the probability of the data given the model (i.e. maximising the likelihood) is equivalent to minimising the sum of the squared residuals, so this is what `lm()` actually does since it is computationally simpler than computing the likelihood

To best illustrate the OLS concept, we need to simulate data a little more spread out

```
set.seed(123)
Alien2 <- tibble(humans_eaten = 1:12, intercept = 50, slope = 1.5,
                 size = rnorm(n = 12, mean = intercept + slope*humans_eaten, sd = 4))
fit_alien2_lm <- lm(size ~ humans_eaten, data = Alien2)
```



Residuals on best fit



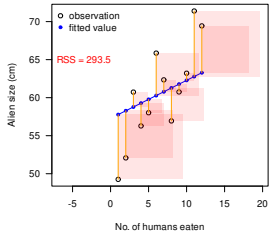
Squared residuals on best fit

LM — model fitting: interlude on ordinary least squares (OLS)

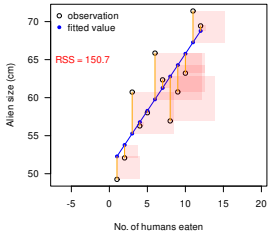
In LM (but not GLM), finding the estimates that maximise the probability of the data given the model (i.e. maximising the likelihood) is equivalent to minimising the sum of the squared residuals, so this is what `lm()` actually does since it is computationally simpler than computing the likelihood

To best illustrate the OLS concept, we need to simulate data a little more spread out

```
set.seed(123)
Alien2 <- tibble(humans_eaten = 1:12, intercept = 50, slope = 1.5,
                 size = rnorm(n = 12, mean = intercept + slope*humans_eaten, sd = 4))
fit_alien2_lm <- lm(size ~ humans_eaten, data = Alien2)
```



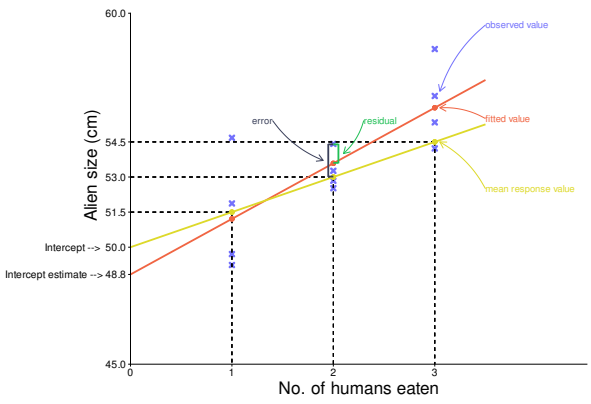
Squared residuals on bad fit



Squared residuals on best fit

LM — model fitting: fit vs ground truth, & vocabulary

```
Alien |> # we are back to using our original data & fit!
  mutate(mean_response_values = intercept + slope * humans_eaten, # truth in population
         fitted_values = fitted(fit_alien_lm),
         residuals = residuals(fit_alien_lm)) -> Alien
```



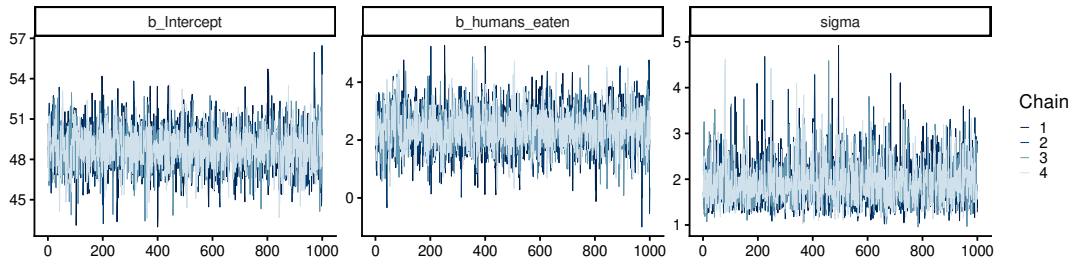
- a *prediction* is the average value of the response given by the fit for a large number of observations with the same predictor characteristics (i.e. any point on the coral line is a prediction)
- a *fitted value* is a *prediction* at observed predictor characteristics (i.e. only the points for 1, 2 and 3 humans on the coral line)
- a *residual* is a difference between an observation and its fitted value (i.e. $\text{observation}_i = \text{fitted}_i + \text{residual}_i$)

LM — model fitting: Bayesian approach with {brms}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Let's first use {brms}, which is simpler than {nimble} but handles a smaller set of models

```
library(brms) # Bayesian regression models using stan
fit_alien_lm_brms <- brm(size ~ humans_eaten, data = Alien)
mcmc_plot(fit_alien_lm_brms, type = "trace") # many diagnostic plots can be produced
```

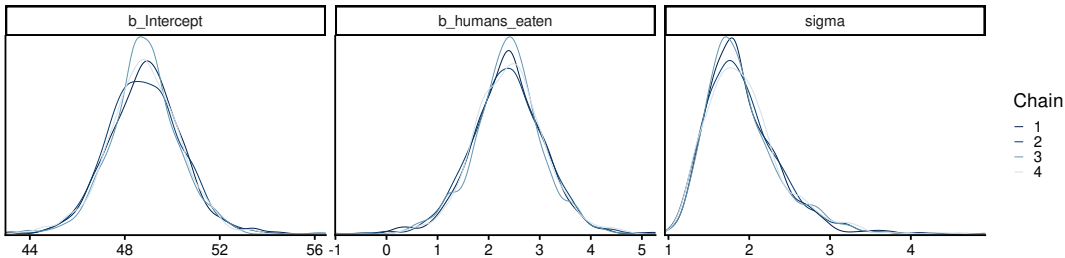


LM — model fitting: Bayesian approach with {brms}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Here are the posteriors distributions

```
mcmc_plot(fit_alien_lm_brms, type = "dens_overlay") # many diagnostic plots can be produced
```



LM — model fitting: Bayesian approach with {brms}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Here is the summary for the posteriors distributions

```
summary(fit_alien_lm_brms) # or fixef(fit_alien_lm_brms) for a focus on coefficients
```

```
 Family: gaussian
  Links: mu = identity
Formula: size ~ humans_eaten
   Data: Alien (Number of observations: 12)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept       48.83      1.51    45.81    51.85 1.00     3306     2454
humans_eaten     2.38      0.70     0.99     3.76 1.00     3166     2281

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     1.92      0.46     1.26     3.03 1.00     2388     2555

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

LM — model fitting: Bayesian approach with {nimble}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Let's now use {nimble}

```
library(nimble)
library(posterior)
library(bayesplot)

code <- nimbleCode({
  for (i in seq_len(N)) { # model mimicking the data generating process
      size[i] ~ dnorm(mean = intercept + slope*humans_eaten[i], sd = sigma)
  }
  # priors
  intercept ~ dnorm(mean = 50, sd = 5)
  slope ~ dnorm(mean = 0, sd = 5)
  sigma ~ dunif(min = 0, max = 5)
})

constants <- list(N = nrow(Alien), humans_eaten = Alien$humans_eaten)

inits <- list(intercept = mean(Alien$size), slope = 0, sigma = 1)
```

LM — model fitting: prior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Before fitting the model, let's make sure our priors are not too vague

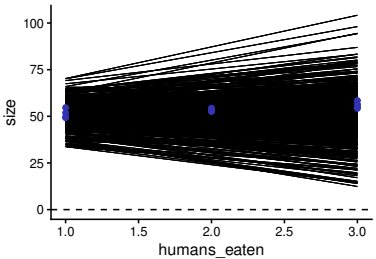
```
set.seed(123)
n_sims <- 1000

prior <- tibble(sim = seq_len(n_sims),
                intercept = rnorm(n_sims, mean = 50, sd = 5),
                slope = rnorm(n_sims, mean = 0, sd = 5),
                sigma = runif(n_sims, min = 0, max = 5))

humans_eaten_possible <- min(Alien$humans_eaten):max(Alien$humans_eaten)

expand_grid(prior, humans_eaten = humans_eaten_possible) |>
  mutate(size = intercept[sim] + slope[sim]*humans_eaten,
         resp = size + rnorm(n = n(), 0, sigma[sim])) -> prior_pred

ggplot(prior_pred, aes(x = humans_eaten, y = size)) +
  geom_line(aes(group = sim), linewidth = 0.1) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point(aes(group = NULL), colour = "#3333B3", data = Alien)
```

LM — model fitting: prior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Before fitting the model, let's make sure our priors are not too vague

```
set.seed(123)
n_sims <- 1000

prior <- tibble(sim = seq_len(n_sims),
                intercept = rnorm(n_sims, mean = 50, sd = 5),
                slope = rnorm(n_sims, mean = 0, sd = 5),
                sigma = runif(n_sims, min = 0, max = 5))

humans_eaten_possible <- min(Alien$humans_eaten):max(Alien$humans_eaten)

expand_grid(prior, humans_eaten = humans_eaten_possible) |>
  mutate(size = intercept[sim] + slope[sim]*humans_eaten,
         resp = size + rnorm(n = n(), 0, sigma[sim])) -> prior_pred

ggplot(prior_pred, aes(x = humans_eaten, y = size)) +
  geom_line(aes(group = sim), linewidth = 0.1) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point(aes(group = NULL), colour = "#3333B3", data = Alien)
```



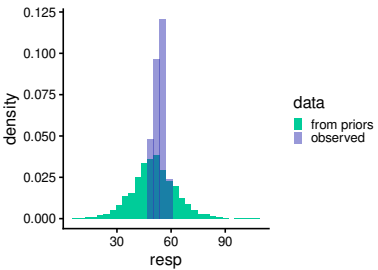
- priors for intercept and slope look OK: wide enough to cover a wide range of relationships & not too wide
- scaling the data would improve things (see later)

LM — model fitting: prior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Before fitting the model, let's make sure our priors are not too vague

```
ggplot() +
  geom_histogram(aes(x = resp, after_stat(density), fill = "from priors"),
                 data = prior_pred) +
  geom_histogram(aes(x = size, after_stat(density), fill = "observed"),
                 alpha = 0.5, data = Alien) +
  scale_fill_manual(values = c("#00CC99", "#3333B3"), name = "data")
```



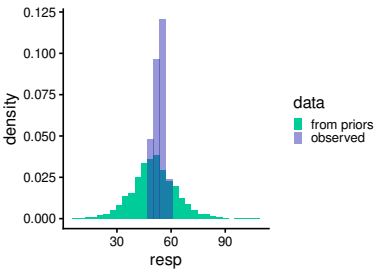
- priors for sigma look OK too: potentially the priors can produce data encompassing the real ones, without risking producing silly alien sizes

LM — model fitting: prior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Before fitting the model, let's make sure our priors are not too vague

```
ggplot() +
  geom_histogram(aes(x = resp, after_stat(density), fill = "from priors"),
                 data = prior_pred) +
  geom_histogram(aes(x = size, after_stat(density), fill = "observed"),
                 alpha = 0.5, data = Alien) +
  scale_fill_manual(values = c("#00CC99", "#3333B3"), name = "data")
```



NB:

- one should not fine tune the priors based on the observed data since a dataset may be biased (risk of *overfitting*), but one should make sure that the priors are somewhat realistic
- how to best choose priors is an ongoing research question, and the most covered topic in Bayesian statistics (e.g. more objective technics and databases are starting to emerge)
- if you have enough data, also long as priors are not too silly, they should have little effect on your posteriors (and you can check that)
- whatever you do, do describe what you do in your papers!

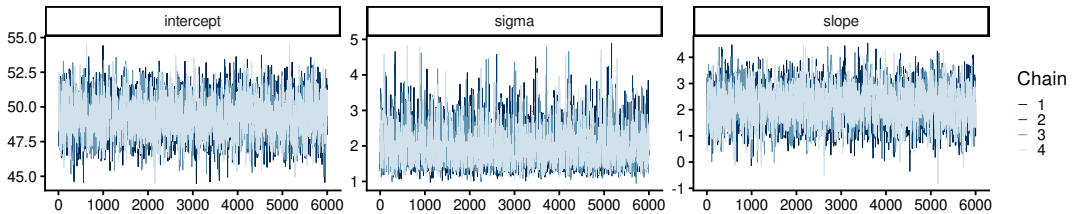
- priors for sigma look OK too: potentially the priors can produce data encompassing the real ones, without risking producing silly alien sizes

LM — model fitting: Bayesian approach with {nimble}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

We can now fit the model with {nimble}

```
fit_alien_lm_nimb <- nimbleMCMC(code = code,
                                constants = constants,
                                data = list(size = Alien$size), # response goes here
                                inits = inits, nchains = 4, setSeed = 1:4,
                                nburnin = 4000) # amply sufficient here (brms used 1000)
mcmc_trace(fit_alien_lm_nimb) # not mcmc_plot which only is for {brms}
```

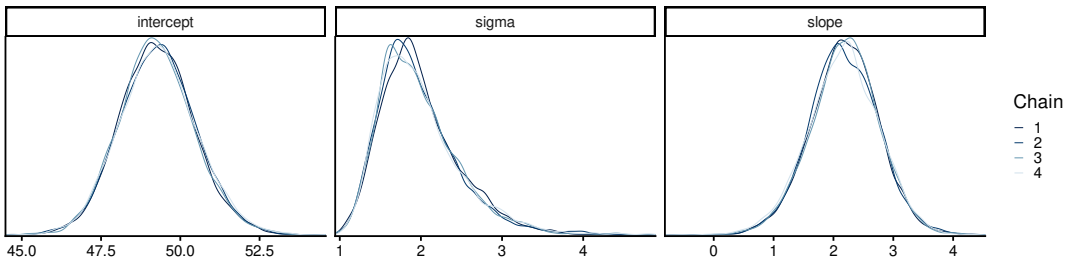


LM — model fitting: Bayesian approach with {nimble}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Here are the posterior distributions

```
mcmc_dens_overlay(fit_alien_lm_nimb)
```



LM — model fitting: Bayesian approach with {nimble}

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Here is the summary for the posteriors distributions

```
fit_alien_lm_nimb_d <- as_draws(fit_alien_lm_nimb)
summary(fit_alien_lm_nimb_d)
```

```
# A tibble: 3 x 10
  variable   mean median    sd   mad    q5   q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
1 intercept 49.3   49.3  1.20  1.17  47.4  51.3   1.00    3036.    6449.
2 sigma      1.98   1.89 0.502 0.439  1.33  2.93  1.00    3722.    4248.
3 slope      2.18   2.18 0.582 0.564  1.21  3.12  1.00    3042.    6209.
```

LM — model fitting: posterior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

After fitting the model, let's make sure our posterior are not off

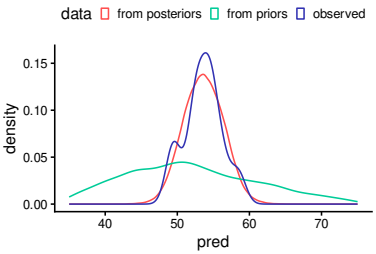
```
posterior1 <- tibble(
  intercept = fit_alien_lm_nimb_d[, 1, "intercept", drop = TRUE],
  slope = fit_alien_lm_nimb_d[, 1, "slope", drop  = TRUE],
  sigma = fit_alien_lm_nimb_d[, 1, "sigma", drop = TRUE])

expand_grid(posterior1, humans_eaten = Alien$humans_eaten) |>
  mutate(error = rnorm(n = n(), mean = 0, sd = sigma),
         pred = intercept + slope * humans_eaten + error) -> pred1

ggplot() +
  geom_density(aes(x = pred, after_stat(density),
                   colour = "from posteriors"), data = pred1) +
  geom_density(aes(x = resp, after_stat(density),
                   colour = "from priors"), data = prior_pred,) +
  geom_density(aes(x = size, after_stat(density),
                   colour = "observed"), data = Alien) +
  scale_colour_manual(values = c("#FF5050", "#00CC99", "#3333B3"),
                      name = "data",
                      guide = guide_legend(position = "top")) +
  xlim(limits = c(35, 75))
```

LM — model fitting: posterior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

After fitting the model, let's make sure our posterior are not off

```
posterior1 <- tibble(
  intercept = fit_alien_lm_nimb_d[, 1, "intercept", drop = TRUE],
  slope = fit_alien_lm_nimb_d[, 1, "slope", drop  = TRUE],
  sigma = fit_alien_lm_nimb_d[, 1, "sigma", drop = TRUE])

expand_grid(posterior1, humans_eaten = Alien$humans_eaten) |>
  mutate(error = rnorm(n = n(), mean = 0, sd = sigma),
         pred = intercept + slope * humans_eaten + error) -> pred1

ggplot() +
  geom_density(aes(x = pred, after_stat(density),
                   colour = "from posteriors"), data = pred1) +
  geom_density(aes(x = resp, after_stat(density),
                   colour = "from priors"), data = prior_pred,) +
  geom_density(aes(x = size, after_stat(density),
                   colour = "observed"), data = Alien) +
  scale_colour_manual(values = c("#FF5050", "#00CC99", "#3333B3"),
                      name = "data",
                      guide = guide_legend(position = "top")) +
  xlim(limits = c(35, 75))
```



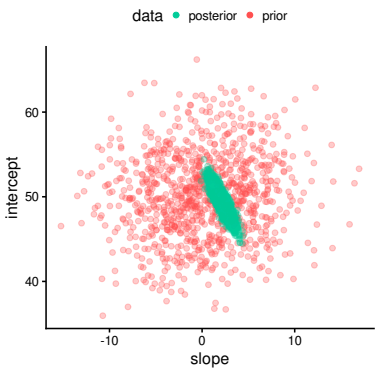
- this looks OK: the posterior predictions are much more concentrated than the prior ones, and close to the observations

LM — model fitting: posterior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Let's check how correlated posteriors are

```
ggplot() +
  geom_point(aes(y = intercept, x = slope, colour = "prior"),
             alpha = 0.3, data = prior) +
  geom_point(aes(y = intercept, x = slope, colour = "posterior"),
             alpha = 0.3, data = posterior1) +
  scale_colour_manual(
    values = c("#00CC99", "#FF5050"),
    name = "data",
    guide = guide_legend(position = "top",
                         override.aes = list(alpha = 1)))
```



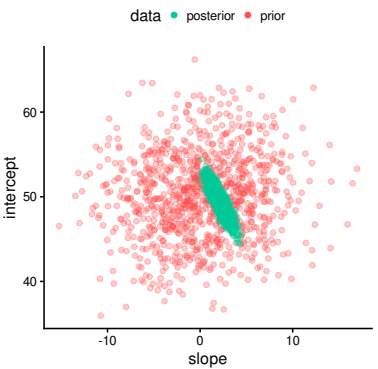
- this looks OK: the posterior predictions are within the priors, which further validate the choice of our priors

LM — model fitting: posterior predictive checking

- in Bayesian statistics, *fitting the model* means using the data to characterise the posterior distributions

Let's check how correlated posteriors are

```
ggplot() +
  geom_point(aes(y = intercept, x = slope, colour = "prior"),
             alpha = 0.3, data = prior) +
  geom_point(aes(y = intercept, x = slope, colour = "posterior"),
             alpha = 0.3, data = posterior1) +
  scale_colour_manual(
    values = c("#00CC99", "#FF5050"),
    name = "data",
    guide = guide_legend(position = "top",
                         override.aes = list(alpha = 1)))
```



NB: for real applications

- the predictor variables should be scaled to avoid strong correlations (see later)

- this looks OK: the posterior predictions are within the priors, which further validate the choice of our priors

LM — model fitting results: estimates compared

Let's compare the results

`lm()`

```
# A tibble: 3 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     48.8         1.35   45.8      51.8
2 humans_eaten     2.37        0.623   0.987     3.76
3 sigma            1.76        1.39    1.01      6.36
```

`brms::brm()`

```
# A tibble: 3 x 5
  Parameter    Estimate Est.Error   Q2.5 Q97.5
  <chr>           <dbl>     <dbl>  <dbl> <dbl>
1 Intercept       48.8      1.51   45.8   51.8
2 humans_eaten     2.38     0.700   0.993  3.76
3 sigma            1.92     0.463   1.26   3.03
```

`nimble::nimbleMCMC()`

```
# A tibble: 3 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept 49.3  1.20  47.0     51.7
2 slope      2.18 0.582  0.997    3.30
3 sigma      1.98 0.502  1.26     3.22
```

LM — model fitting results: estimates compared

Let's compare the results

`lm()`

```
# A tibble: 3 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     48.8         1.35   45.8      51.8
2 humans_eaten     2.37        0.623   0.987     3.76
3 sigma            1.76        1.39    1.01      6.36
```

`brms::brm()`

```
# A tibble: 3 x 5
  Parameter    Estimate Est.Error   Q2.5 Q97.5
  <chr>           <dbl>     <dbl>  <dbl> <dbl>
1 Intercept       48.8      1.51   45.8   51.8
2 humans_eaten     2.38     0.700   0.993  3.76
3 sigma            1.92     0.463   1.26   3.03
```

`nimble::nimbleMCMC()`

```
# A tibble: 3 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept 49.3  1.20  47.0     51.7
2 slope      2.18 0.582  0.997    3.30
3 sigma      1.98 0.502  1.26     3.22
```

- despite the small sample size (N=12) the estimates are in agreement and all intervals include ground truth values

LM — model fitting results: estimates compared

Let's compare the results

`lm()`

```
# A tibble: 3 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     48.8         1.35   45.8      51.8
2 humans_eaten     2.37        0.623   0.987     3.76
3 sigma            1.76        1.39    1.01      6.36
```

`brms::brm()`

```
# A tibble: 3 x 5
  Parameter    Estimate Est.Error   Q2.5 Q97.5
  <chr>           <dbl>     <dbl>  <dbl> <dbl>
1 Intercept       48.8      1.51   45.8   51.8
2 humans_eaten     2.38     0.700   0.993  3.76
3 sigma            1.92     0.463   1.26   3.03
```

`nimble::nimbleMCMC()`

```
# A tibble: 3 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept 49.3  1.20  47.0     51.7
2 slope      2.18 0.582  0.997    3.30
3 sigma      1.98 0.502  1.26     3.22
```

- despite the small sample size (N=12) the estimates are in agreement and all intervals include ground truth values
- we never obtained 50 (intercept), 1.5 (slope), or 2 (sigma) because a sample is unlikely to reflect perfectly the population-level values; only N=∞ or talking to the goddess could get you those

LM — model fitting results: estimates compared

Let's compare the results

`lm()`

```
# A tibble: 3 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     48.8         1.35    45.8      51.8
2 humans_eaten     2.37        0.623    0.987     3.76
3 sigma            1.76        1.39     1.01      6.36
```

`brms::brm()`

```
# A tibble: 3 x 5
  Parameter    Estimate Est.Error   Q2.5 Q97.5
  <chr>           <dbl>     <dbl>  <dbl> <dbl>
1 Intercept       48.8      1.51   45.8   51.8
2 humans_eaten     2.38     0.700   0.993  3.76
3 sigma            1.92     0.463   1.26   3.03
```

`nimble::nimbleMCMC()`

```
# A tibble: 3 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept 49.3  1.20  47.0     51.7
2 slope      2.18 0.582  0.997    3.30
3 sigma      1.98 0.502  1.26     3.22
```

- despite the small sample size (N=12) the estimates are in agreement and all intervals include ground truth values
- we never obtained 50 (intercept), 1.5 (slope), or 2 (sigma) because a sample is unlikely to reflect perfectly the population-level values; only N=∞ or talking to the goddess could get you those

Geeky notes:

- in `lm()` $\hat{\sigma}$ is usually not a parameter that is estimated directly, it is deduced once the intercept and slope(s) have been fitted (behind the scenes I struggled to compute the SE and CI for it)

LM — model fitting results: estimates compared

Let's compare the results

`lm()`

```
# A tibble: 3 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     48.8         1.35   45.8      51.8
2 humans_eaten     2.37        0.623   0.987     3.76
3 sigma            1.76        1.39    1.01      6.36
```

`brms::brm()`

```
# A tibble: 3 x 5
  Parameter    Estimate Est.Error   Q2.5 Q97.5
  <chr>           <dbl>     <dbl>  <dbl> <dbl>
1 Intercept       48.8      1.51   45.8   51.8
2 humans_eaten     2.38     0.700  0.993  3.76
3 sigma            1.92     0.463  1.26   3.03
```

`nimble::nimbleMCMC()`

```
# A tibble: 3 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept 49.3  1.20  47.0     51.7
2 slope      2.18 0.582  0.997    3.30
3 sigma      1.98 0.502  1.26     3.22
```

- despite the small sample size (N=12) the estimates are in agreement and all intervals include ground truth values
- we never obtained 50 (intercept), 1.5 (slope), or 2 (sigma) because a sample is unlikely to reflect perfectly the population-level values; only N=∞ or talking to the goddess could get you those

Geeky notes:

- in `lm()` $\hat{\sigma}$ is usually not a parameter that is estimated directly, it is deduced once the intercept and slope(s) have been fitted (behind the scenes I struggled to compute the SE and CI for it)
- by default nimble returns q5 and q95 (not q2.5 and q97.5), so I had to request this manually (see `?summarise_draws`)

Practice 2

1. explore the dataset `penguins` (readily available in R)
2. do check its documentation `?penguins`
3. using this dataset, fit a linear model with:

- `body_mass` as the response variables
- `flipper_len` as predictors

Interpreting coefficients

Interpreting coefficients — introduction

Interpreting the meaning of *regression coefficients* (= *parameter estimates*) correctly in statistical models is the most critical part given that:

- coefficients mediate the relationship between the statistics and the biology

Interpreting coefficients — introduction

Interpreting the meaning of *regression coefficients* (= *parameter estimates*) correctly in statistical models is the most critical part given that:

- coefficients mediate the relationship between the statistics and the biology
- tests, SE, confidence intervals and prediction intervals, all focus on these coefficients

Interpreting coefficients — introduction

Interpreting the meaning of *regression coefficients* (= *parameter estimates*) correctly in statistical models is the most critical part given that:

- coefficients mediate the relationship between the statistics and the biology
- tests, SE, confidence intervals and prediction intervals, all focus on these coefficients
- different software and packages may influence aspects (contrasts, scale) that impact the meaning of the coefficients

Interpreting coefficients — introduction

Interpreting the meaning of *regression coefficients* (= *parameter estimates*) correctly in statistical models is the most critical part given that:

- coefficients mediate the relationship between the statistics and the biology
- tests, SE, confidence intervals and prediction intervals, all focus on these coefficients
- different software and packages may influence aspects (contrasts, scale) that impact the meaning of the coefficients
- most scientists fail to do so correctly, jeopardising the conclusions drawn in many publications

Interpreting coefficients — introduction

Interpreting the meaning of *regression coefficients* (= *parameter estimates*) correctly in statistical models is the most critical part given that:

- coefficients mediate the relationship between the statistics and the biology
- tests, SE, confidence intervals and prediction intervals, all focus on these coefficients
- different software and packages may influence aspects (contrasts, scale) that impact the meaning of the coefficients
- most scientists fail to do so correctly, jeopardising the conclusions drawn in many publications

NB:

- it requires some focus, but no more than middle school mathematics knowledge

Interpreting coefficients — data preparation: clean up

Let's create a single clean dataset with all the variables we will need for all models

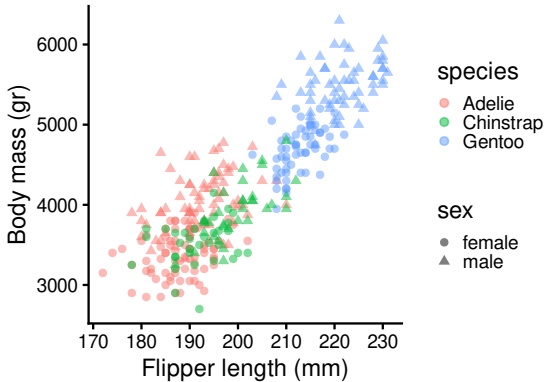
```
penguins |>
  as_tibble() |>
  select(-starts_with("bill")) |>
  drop_na() |>
  mutate(flipper_len_z = scale(flipper_len)[,1], .after = flipper_len) |>
  mutate(body_mass_z = scale(body_mass)[,1], .after = body_mass) -> penguins_clean

penguins_clean
```

```
# A tibble: 333 x 8
  species island    flipper_len flipper_len_z body_mass body_mass_z sex     year
  <fct>   <fct>           <int>         <dbl>     <int>       <dbl> <fct>  <int>
1 Adelie  Torgersen         181        -1.42       3750      -0.568 male    2007
2 Adelie  Torgersen         186        -1.07       3800      -0.506 female  2007
3 Adelie  Torgersen         195        -0.426      3250      -1.19  female  2007
4 Adelie  Torgersen         193        -0.568      3450      -0.940 female  2007
5 Adelie  Torgersen         190        -0.782      3650      -0.692 male    2007
# i 328 more rows
```

NB: `scale()` creates z-scores (it removes the mean and divide by the SD)

Interpreting coefficients — data preparation: plot

```
ggplot(penguins_clean) +
  aes(y = body_mass, x = flipper_len, colour = species, shape = sex) +
  geom_point(alpha = 0.5) +
  labs(y = "Body mass (gr)", x = "Flipper length (mm)")
```



Interpreting coefficients — intercept and slopes

How to interpret the regression coefficients?

```
fit_flipper <- lm(body_mass ~ flipper_len, data = penguins_clean)
coef(fit_flipper)
```

```
(Intercept) flipper_len
-5872.09268    50.15327
```

Interpreting coefficients — intercept and slopes

How to interpret the regression coefficients?

```
fit_flipper <- lm(body_mass ~ flipper_len, data = penguins_clean)
coef(fit_flipper)
```

```
(Intercept) flipper_len
-5872.09268    50.15327
```

- the *intercept* is the prediction when all regressors are at 0, so here it corresponds to the average body mass of a penguin that would have no flippers
  (i.e. a nonsensical *extrapolation* in this case)

```
predict(fit_flipper, newdata = tibble(flipper_len = 0))
```

```
       1
-5872.093
```

Interpreting coefficients — intercept and slopes

How to interpret the regression coefficients?

```
fit_flipper <- lm(body_mass ~ flipper_len, data = penguins_clean)
coef(fit_flipper)
```

```
(Intercept) flipper_len
-5872.09268    50.15327
```

- the *slope* gives the change in predicted values when its associated regressor increases by 1 unit, so here it means that the model predicts that penguins with flippers 1mm longer than others are on average 50gr heavier than such others

```
pred_diff <- predict(fit_flipper, newdata = tibble(flipper_len = c(200, 201)))
pred_diff
```

```
       1        2
4158.561 4208.714
```

```
pred_diff[2] - pred_diff[1] # or simply diff(pred_diff)
```

```
       2
50.15327
```

Interpreting coefficients — scaling predictors

How to interpret the regression coefficients when predictors are scaled?

```
fit_flipper_z <- lm(body_mass ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_z)
```

```
  (Intercept) flipper_len_z
    4207.0571      702.9364
```

Interpreting coefficients — scaling predictors

How to interpret the regression coefficients when predictors are scaled?

```
fit_flipper_z <- lm(body_mass ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_z)
```

```
  (Intercept) flipper_len_z
    4207.0571      702.9364
```

NB:

- *scaling* means removing the mean (i.e. *centering*) and dividing by the SD (i.e. *standardising*)
- scaling both the response variable and the predictors is a recommended practice when using {nimble}, {stan}, etc… since it makes the definition of priors more consistent across studies; it also improves the efficiency of some MCMC samplers since it decreases correlations between posteriors
- some packages do similar things internally (e.g. {brms}), while doing themselves the necessary *back transformation* for allowing interpretations at the original scales

Interpreting coefficients — scaling predictors

How to interpret the regression coefficients when predictors are scaled?

```
fit_flipper_z <- lm(body_mass ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_z)
```

```
(Intercept) flipper_len_z
  4207.0571      702.9364
```

- the *intercept* remains the prediction when all regressors are at 0, but `flipper_len = 0` meant a flipper length of 0mm, while `flipper_len_z = 0` means an average flipper length; so here the intercept corresponds to the average body mass of a penguin that would have flippers of 201mm (i.e. no longer nonsensical)

Interpreting coefficients — scaling predictors

How to interpret the regression coefficients when predictors are scaled?

```
fit_flipper_z <- lm(body_mass ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_z)
```

```
  (Intercept) flipper_len_z
    4207.0571      702.9364
```

- the *slope* remains the change in predicted values when its associated regressor increases by 1 unit, but here 1 unit is not longer 1mm but 1 SD in flipper length; so here it means that the model predicts that penguins with flippers 1 SD longer than others (14mm) are on average 703gr heavier than such others

back transformation

It is possible to convert the coefficients to get back to the original (untransformed) scale!

```
coef(fit_flipper_z)["flipper_len_z"] / sd(penguins_clean$flipper_len)
```

```
flipper_len_z
     50.15327
```

Interpreting coefficients — scaling the response

How to interpret the regression coefficients when the response is scaled?

```
fit_flipper_bmz <- lm(body_mass_z ~ flipper_len, data = penguins_clean)
coef(fit_flipper_bmz)
```

```
(Intercept) flipper_len
-12.5173273   0.0622855
```

- again, the *intercept* remains the prediction when all regressors are at 0, and the *slope* remains the change in predicted values when its associated regressor increases by 1 unit, but this time the predicted values are expressed in numbers of SDs above the mean (i.e. z-scores) rather than grams

back transformation

```
coef(fit_flipper_bmz)["(Intercept)"] * sd(penguins_clean$body_mass) + mean(penguins_clean$body_mass)
```

```
(Intercept)
  -5872.093
```

```
coef(fit_flipper_bmz)["flipper_len"] * sd(penguins_clean$body_mass)
```

```
flipper_len
   50.15327
```

Practice 3

How to interpret the regression coefficients when the response and predictors are scaled?

```
fit_flipper_zz <- lm(body_mass_z ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_zz)
```

```
  (Intercept) flipper_len_z
-2.689131e-16  8.729789e-01
```

1. interpret the slope on its new scale
2. interpret the intercept on its new scale
3. apply the required back transformation to perform the interpretation of the slope on the original scale
4. think about how you would define the priors in this case

Solution

How to interpret the regression coefficients when the response and predictors are scaled?

```
fit_flipper_zz <- lm(body_mass_z ~ flipper_len_z, data = penguins_clean)
coef(fit_flipper_zz)
```

```
  (Intercept) flipper_len_z
-2.689131e-16  8.729789e-01
```

1. the slope describes the increase in SD of body mass for an increase of 1 SD in flipper length
2. the intercept is the mean of the body mass z-scores for a penguin with average flipper length, which is why it is 0 (a group of penguins with an average flipper length is expected to have an average body mass)
3. back transformation of the slope:

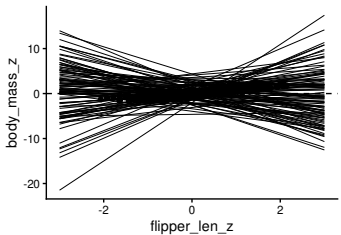
```
(coef(fit_flipper_zz)["flipper_len_z"] / sd(penguins_clean$flipper_len))*sd(penguins_clean$body_mass)
```

```
flipper_len_z
     50.15327
```

Solution

4. something like $\mathcal{N}(0, \sigma = 2)$ for both the intercept and slope priors seems appropriate (& close to {brms} default settings)

```
set.seed(123)
n_sims <- 100

prior <- tibble(sim = seq_len(n_sims),
                intercep = rnorm(n_sims, mean = 0, sd = 2),
                slope = rnorm(n_sims, mean = 0, sd = 2))

flipper_len_z_possible <- -3:3

expand_grid(prior, flipper_len_z = flipper_len_z_possible) |>
  mutate(
    body_mass_z = intercept[sim] + slope[sim]*flipper_len_z) -> prior_pred

ggplot(prior_pred, aes(flipper_len_z, body_mass_z)) +
  geom_line(aes(group = sim), linewidth = 0.1) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point(aes(group = NULL), data = penguins_clean)
```



Interpreting coefficients — qualitative predictors

How to interpret the regression coefficients?

```
fit_sex <- lm(body_mass ~ sex, data = penguins_clean)
coef(fit_sex)
```

```
(Intercept)     sexmale
  3862.2727    683.4118
```

Interpreting coefficients — qualitative predictors

How to interpret the regression coefficients?

```
fit_sex <- lm(body_mass ~ sex, data = penguins_clean)
coef(fit_sex)
```

```
(Intercept)     sexmale
  3862.2727    683.4118
```

Any idea?

Interpreting coefficients — qualitative predictors

How to interpret the regression coefficients?

```
fit_sex <- lm(body_mass ~ sex, data = penguins_clean)
coef(fit_sex)
```

```
(Intercept)     sexmale
  3862.2727    683.4118
```

Any idea?

NB: it is usually best to avoid lucky guesses in this context

Interpreting coefficients — model matrix interlude

It is more useful to check how the data are converted into the *model matrix*:

```
penguins_clean # data
```

```
# A tibble: 333 x 8
  species island    flipper_len flipper_len_z body_mass body_mass_z sex     year
  <fct>   <fct>           <int>         <dbl>     <int>       <dbl> <fct>  <int>
1 Adelie  Torgersen         181        -1.42       3750      -0.568 male    2007
2 Adelie  Torgersen         186        -1.07       3800      -0.506 female  2007
3 Adelie  Torgersen         195        -0.426      3250      -1.19  female  2007
4 Adelie  Torgersen         193        -0.568      3450      -0.940 female  2007
5 Adelie  Torgersen         190        -0.782      3650      -0.692 male    2007
# i 328 more rows
```

```
head(model.matrix(fit_sex))
```

```
  (Intercept) sexmale
1           1       1
2           1       0
3           1       0
4           1       0
5           1       1
6           1       0
```

Interpreting coefficients — model matrix interlude

It is more useful to check how the data are converted into the *model matrix*:

```
penguins_clean # data
```

```
# A tibble: 333 x 8
  species island    flipper_len flipper_len_z body_mass body_mass_z sex     year
  <fct>   <fct>           <int>         <dbl>     <int>       <dbl> <fct>  <int>
1 Adelie  Torgersen         181        -1.42       3750      -0.568 male    2007
2 Adelie  Torgersen         186        -1.07       3800      -0.506 female  2007
3 Adelie  Torgersen         195        -0.426      3250      -1.19  female  2007
4 Adelie  Torgersen         193        -0.568      3450      -0.940 female  2007
5 Adelie  Torgersen         190        -0.782      3650      -0.692 male    2007
# i 328 more rows
```

```
head(model.matrix(fit_sex))
```

```
  (Intercept) sexmale
1           1       1
2           1       0
3           1       0
4           1       0
5           1       1
6           1       0
```

- the model matrix is multiplied by the regression coefficients, so the coefficient `sexmale` is multiplied by 1 when the row corresponds to a male, and by 0 when it corresponds to a female

Interpreting coefficients — model matrix interlude

It is more useful to check how the data are converted into the *model matrix*:

```
penguins_clean # data
```

```
# A tibble: 333 x 8
  species island    flipper_len flipper_len_z body_mass body_mass_z sex     year
  <fct>   <fct>           <int>         <dbl>     <int>       <dbl> <fct>  <int>
1 Adelie  Torgersen         181        -1.42       3750      -0.568 male    2007
2 Adelie  Torgersen         186        -1.07       3800      -0.506 female  2007
3 Adelie  Torgersen         195        -0.426      3250      -1.19  female  2007
4 Adelie  Torgersen         193        -0.568      3450      -0.940 female  2007
5 Adelie  Torgersen         190        -0.782      3650      -0.692 male    2007
# i 328 more rows
```

```
head(model.matrix(fit_sex))
```

```
  (Intercept) sexmale
1           1       1
2           1       0
3           1       0
4           1       0
5           1       1
6           1       0
```

- the model matrix is multiplied by the regression coefficients, so the coefficient `sexmale` is multiplied by 1 when the row corresponds to a male, and by 0 when it corresponds to a female
- by default, R chooses a reference level (by default, the first in alphabetical order) and expresses other effects in relation to that reference, a method called *treatment contrast*

Interpreting coefficients — qualitative predictors

How to interpret the regression coefficients?

```
coef(fit_sex)
```

```
(Intercept)     sexmale
  3862.2727    683.4118
```

predicted value$_i$ = (Intercept) $\times$ 1 + sexmale $\times$ (0 if i is female or 1 if i is male)

- the *intercept* remains the prediction when all regressors are at 0, so here the average body mass of a penguin who is female
- the value called `sexmale` tells you by how much the predicted values increase when the column of the design matrix called `sexmale` changes from 0 to 1 $\rightarrow$ it is thus the average increase in body mass of male penguins compared to female penguins

When in doubt, do compare predictions!

```
pred_sex <- predict(fit_sex, newdata = tibble(sex = c("female", "male")))
pred_sex
```

```
       1        2
3862.273 4545.685
```

Practice 4

How to interpret the regression coefficients with an interaction between a qualitative and quantitative predictor?

```
fit_flipper_int <- lm(body_mass ~ flipper_len*sex, data = penguins_clean)
coef(fit_flipper_int)
```

```
      (Intercept)       flipper_len           sexmale flipper_len:sexmale
-5443.9607499          47.1527260         406.8015395          -0.2942184
```

1. check the design matrix to understand the regressor used by `flipper_len:sexmale`
2. predict by hand:
   - the average body mass of a female penguin with a flipper length of 200 mm
   - the average body mass of a male penguin with a flipper length of 200 mm
3. interpret the meaning of all 4 coefficients
4. does body mass increase with flipper length in males?

Solution

```
coef(fit_flipper_int)
```

```
       (Intercept)         flipper_len             sexmale flipper_len:sexmale
     -5443.9607499          47.1527260         406.8015395          -0.2942184
```

1. the design matrix tells us that `flipper_len:sexmale` is 0 for a female and the flipper length when the penguin is male

```
head(model.matrix(fit_flipper_int), n = 2)
```

```
  (Intercept) flipper_len sexmale flipper_len:sexmale
1           1         181       1                 181
2           1         186       0                   0
```

```
head(penguins_clean, n = 2)
```

```
# A tibble: 2 x 8
  species island    flipper_len flipper_len_z body_mass body_mass_z sex     year
  <fct>   <fct>           <int>         <dbl>     <int>       <dbl> <fct>  <int>
1 Adelie  Torgersen         181         -1.42      3750      -0.568 male    2007
2 Adelie  Torgersen         186         -1.07      3800      -0.506 female  2007
```

Solution

```
coef(fit_flipper_int)
```

```
      (Intercept)      flipper_len           sexmale flipper_len:sexmale
    -5443.9607499        47.1527260      406.8015395         -0.2942184
```

2. predictions can be computed as follows:

predicted value$_i$ = (Intercept) $\times$ 1 + flipper_len $\times$ flipper_len$_i$ + sexmale $\times$
(0 if i is female or 1 if i is male) + flipper_len:sexmale $\times$ (0 if i is female or flipper_len$_i$ if i is male)

- the average body mass of a female penguin with a flipper length of 200 mm

$$= -5443.96 + 47.15 \times 200 + 406.80 \times 0 - 0.29 \times 0 \sim 3986$$

Solution

```
coef(fit_flipper_int)
```

```
      (Intercept)      flipper_len           sexmale flipper_len:sexmale
    -5443.9607499        47.1527260       406.8015395         -0.2942184
```

2. predictions can be computed as follows:

predicted value$_i$ = (Intercept) $\times 1 +$ flipper_len $\times$ flipper_len$_i$ $+$ sexmale $\times$
(0 if i is female or 1 if i is male) + flipper_len:sexmale $\times$ (0 if i is female or flipper_len$_i$ if i is male)

- the average body mass of a female penguin with a flipper length of 200 mm

$$= -5443.96 + 47.15 \times 200 + 406.80 \times 0 - 0.29 \times 0 \sim 3986$$

- the average body mass of a male penguin with a flipper length of 200 mm

$$= -5443.96 + 47.15 \times 200 + 406.80 \times 1 - 0.29 \times 200 = 3986 \sim 4335$$

Exact solution (without rounding)

```
predict(fit_flipper_int, newdata = tibble(sex = c("female", "male"), flipper_len = 200))
```

```
       1        2
3986.584 4334.542
```

Solution

```
coef(fit_flipper_int)
```

```
      (Intercept)      flipper_len          sexmale flipper_len:sexmale
    -5443.9607499        47.1527260       406.8015395        -0.2942184
```

3. interpretation

- `(Intercept)` = average mean body mass of females with flippers of 0mm
- `flipper_len` = average increase in body mass for females when flipper length is increased by 1mm
- `sexmale` = average increase in male body mass compared to female body mass for penguins with flippers of 0mm
- `flipper_len:sexmale` = average increase in male body mass compared to the increase in females when flipper length is increased by 1mm

Solution

```
coef(fit_flipper_int)
```

```
       (Intercept)        flipper_len            sexmale flipper_len:sexmale
     -5443.9607499         47.1527260        406.8015395         -0.2942184
```

4. body mass does increase with flipper length in males since the effect of flipper length in males is 47.15 - 0.29 ($>>$ 0)

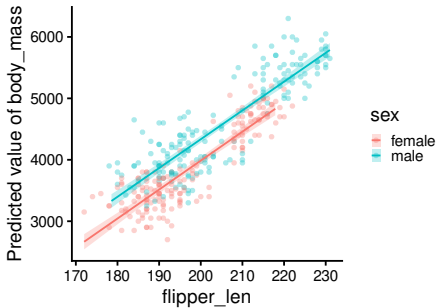
```
predict(fit_flipper_int, newdata = tibble(sex = "male", flipper_len = c(200, 201)))
```

```
       1        2
4334.542 4381.401
```

Interpreting coefficients — plotting predictions

Computing and plotting prediction can be difficult, but the package {modelbased} from the {easystats} universe (easystats.github.io/easystats) makes this easy for many types of models

```
library(modelbased)
estimate_relation(fit_flipper_int) |>
  plot(show_data = TRUE)
```



Interpreting coefficients — conditional and marginal predictions

How to compute/illustrate model predictions for a subset of predictors?

```
fit_flipper_3var <- lm(body_mass ~ flipper_len + sex + species, data = penguins_clean)
```

Example

you want to compute/illustrate the relationship between `body_mass` & `flipper_len` from the fitted model above, what to do with sex & species?

[1] structural correlations are caused by an unbalanced sampling design

Interpreting coefficients — conditional and marginal predictions

How to compute/illustrate model predictions for a subset of predictors?

```
fit_flipper_3var <- lm(body_mass ~ flipper_len + sex + species, data = penguins_clean)
```

Example

you want to compute/illustrate the relationship between `body_mass` & `flipper_len` from the fitted model above, what to do with sex & species?

- **conditional predictions**: the software fixes each non-focal predictor at a specific value (e.g. the mean for quantitative variables & the most frequent category for the qualitative variables) → easiest, but choice of reference can be arbitrary and the computation is even slightly wrong in models involving non-linear transformation(s) (due to *Jensen's inequality*)

[1] structural correlations are caused by an unbalanced sampling design

Interpreting coefficients — conditional and marginal predictions

How to compute/illustrate model predictions for a subset of predictors?

```
fit_flipper_3var <- lm(body_mass ~ flipper_len + sex + species, data = penguins_clean)
```

Example

you want to compute/illustrate the relationship between `body_mass` & `flipper_len` from the fitted model above, what to do with sex & species?

- **conditional predictions**: the software fixes each non-focal predictor at a specific value (e.g. the mean for quantitative variables & the most frequent category for the qualitative variables) → easiest, but choice of reference can be arbitrary and the computation is even slightly wrong in models involving non-linear transformation(s) (due to *Jensen's inequality*)
- **marginal predictions**: the software computes predictions for different values/levels of the focal and non-focal predictors and then averages those predictions across the non-focal predictors → often better, but it is more difficult to grasp since different alternatives exist to compute the *reference grid* (depending on whether one wants to capture or erase the actual or structural correlations between predictors[1])

[1] structural correlations are caused by an unbalanced sampling design

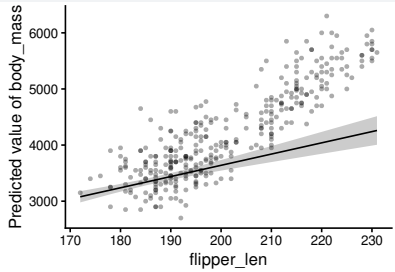
Interpreting coefficients — conditional and marginal predictions

Conditional predictions of the effect of `flipper_len` on `body_mass`

```
cond_pred <- estimate_relation(fit_flipper_3var,
                               by = "flipper_len")
cond_pred
```

```
Model-based Predictions

flipper_len | Predicted |     SE |              95% CI
-----------------------------------------------------
172.00      |   3078.47 |  51.04 | [2978.05, 3178.88]
178.56      |   3209.75 |  37.35 | [3136.28, 3283.22]
185.11      |   3341.01 |  29.68 | [3282.64, 3399.39]
191.67      |   3472.30 |  32.59 | [3408.18, 3536.42]
198.22      |   3603.56 |  44.05 | [3516.91, 3690.21]
204.78      |   3734.84 |  59.28 | [3618.23, 3851.46]
211.33      |   3866.11 |  76.05 | [3716.50, 4015.72]
217.89      |   3997.39 |  93.54 | [3813.38, 4181.41]
224.44      |   4128.65 | 111.41 | [3909.49, 4347.82]
231.00      |   4259.94 | 129.50 | [4005.19, 4514.69]

Variable predicted: body_mass
Predictors modulated: flipper_len
Predictors controlled: sex (female), species (Adelie)
```

```
plot(cond_pred, show_data = TRUE)
```



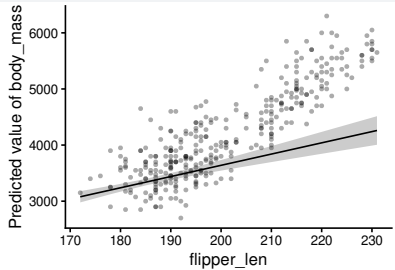
Interpreting coefficients — conditional and marginal predictions

Conditional predictions of the effect of `flipper_len` on `body_mass`

```
cond_pred <- estimate_relation(fit_flipper_3var,
                               by = "flipper_len")
cond_pred
```

```
Model-based Predictions

flipper_len | Predicted |     SE |              95% CI
-----------------------------------------------------
172.00      |   3078.47 |  51.04 | [2978.05, 3178.88]
178.56      |   3209.75 |  37.35 | [3136.28, 3283.22]
185.11      |   3341.01 |  29.68 | [3282.64, 3399.39]
191.67      |   3472.30 |  32.59 | [3408.18, 3536.42]
198.22      |   3603.56 |  44.05 | [3516.91, 3690.21]
204.78      |   3734.84 |  59.28 | [3618.23, 3851.46]
211.33      |   3866.11 |  76.05 | [3716.50, 4015.72]
217.89      |   3997.39 |  93.54 | [3813.38, 4181.41]
224.44      |   4128.65 | 111.41 | [3909.49, 4347.82]
231.00      |   4259.94 | 129.50 | [4005.19, 4514.69]

Variable predicted: body_mass
Predictors modulated: flipper_len
Predictors controlled: sex (female), species (Adelie)
```

```
plot(cond_pred, show_data = TRUE)
```



NB: here, the conditional predictions are given for female Adelie penguins; in this particular model (but not generally), another reference (e.g. male Gentoo penguins) would only shift the regression line up or down, but it would not modify the slope since no interactions are considered and the effects are linear

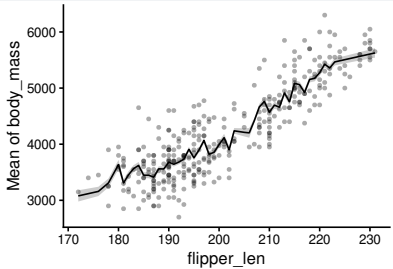
Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("average" method)

```
marg_avg_pred <- estimate_means(
  fit_flipper_3var,
  by = list(flipper_len = penguins_clean$flipper_len),
  estimate = "average")

as_tibble(marg_avg_pred)
```

```
# A tibble: 54 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <int> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172 3078.  51.0  2978.   3179.  60.3   328
2         174 3119.  46.5  3027.   3210.  67.1   328
3         176 3159.  42.2  3075.   3242.  74.8   328
4         178 3309.  40.2  3230.   3388.  82.4   328
5         180 3636.  42.4  3553.   3720.  85.8   328
# i 49 more rows
```

```
plot(marg_avg_pred, show_data = TRUE)
```



NB: here, the marginal prediction is the average of the predicted values for all sampled penguins with the same flipper length and which may differ in their sex or species → preserves correlations between predictors & preserves the distributions of non-focal predictors (i.e. sex & species sampled more weigh more)

Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("average" method)

```
marg_avg_pred <- estimate_means(
  fit_flipper_3var,
  by = list(flipper_len = penguins_clean$flipper_len),
  estimate = "average")

as_tibble(marg_avg_pred)
```

```
# A tibble: 54 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <int> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172 3078.  51.0  2978.   3179.  60.3   328
2         174 3119.  46.5  3027.   3210.  67.1   328
3         176 3159.  42.2  3075.   3242.  74.8   328
4         178 3309.  40.2  3230.   3388.  82.4   328
5         180 3636.  42.4  3553.   3720.  85.8   328
# i 49 more rows
```

```
penguins_clean |>
  select(flipper_len, sex, species) -> grid_avg

grid_avg |>
  mutate(pred = predict(
    fit_flipper_3var,
    newdata = grid_avg)) -> grid_avg

grid_avg |>
  summarise(Mean = mean(pred), .by = "flipper_len") |>
  arrange(flipper_len)
```

```
# A tibble: 54 x 2
  flipper_len  Mean
        <int> <dbl>
1         172 3078.
2         174 3119.
3         176 3159.
4         178 3309.
5         180 3636.
# i 49 more rows
```

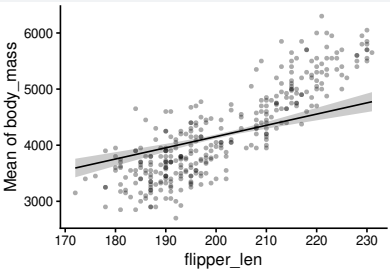
Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("typical" method)

```
marg_typic_pred <- estimate_means(
  fit_flipper_3var,
  by = "flipper_len",
  estimate = "typical")

as_tibble(marg_typic_pred)
```

```
# A tibble: 10 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172  3593.  84.3  3427.   3759.  42.6   328
2         179. 3724.  66.2  3594.   3855.  56.3   328
3         185. 3856.  48.4  3761.   3951.  79.7   328
4         192. 3987.  31.6  3925.   4049. 126.    328
5         198. 4118.  18.8  4081.   4155. 219.    328
# i 5 more rows
```

```
plot(marg_typic_pred, show_data = TRUE)
```



NB: here, the marginal prediction is the average of the predicted values considering that any given flipper length could be found in either sex and in any species with the same probability → breaks down correlations between predictors & flattens the distributions of non-focal predictors (i.e. both sex and all species exert the same weight, as in a perfectly balanced experimental design)

Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("typical" method)

```
marg_typic_pred <- estimate_means(
  fit_flipper_3var,
  by = "flipper_len",
  estimate = "typical")

as_tibble(marg_typic_pred)
```

```
# A tibble: 10 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172  3593.  84.3  3427.   3759.  42.6   328
2         179. 3724.  66.2  3594.   3855.  56.3   328
3         185. 3856.  48.4  3761.   3951.  79.7   328
4         192. 3987.  31.6  3925.   4049. 126.    328
5         198. 4118.  18.8  4081.   4155. 219.    328
# i 5 more rows
```

```
expand_grid(
  flipper_len = seq(172, 231, length = 10),
  sex = unique(penguins_clean$sex),
  species = unique(penguins_clean$species)) -> grid_typic

grid_typic |>
  mutate(pred = predict(
    fit_flipper_3var,
    newdata = grid_typic)) -> grid_typic

grid_typic |>
  summarise(Mean = mean(pred), .by = "flipper_len") |>
  arrange(flipper_len)
```

```
# A tibble: 10 x 2
  flipper_len  Mean
        <dbl> <dbl>
1         172  3593.
2         179. 3724.
3         185. 3856.
4         192. 3987.
5         198. 4118.
# i 5 more rows
```

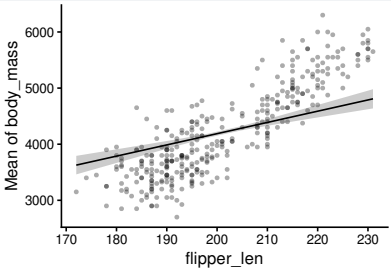
Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("population" method)

```
marg_pop_pred <- estimate_means(
  fit_flipper_3var,
  by = "flipper_len",
  estimate = "population")

as_tibble(marg_pop_pred)
```

```
# A tibble: 10 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172  3627.  84.0  3462.   3792.  43.2   328
2         179. 3758.  65.8  3629.   3888.  57.1   328
3         185. 3890.  47.9  3795.   3984.  81.1   328
4         192. 4021.  31.0  3960.   4082. 130.    328
5         198. 4152.  18.0  4117.   4187. 231.    328
# i 5 more rows
```

```
plot(marg_pop_pred, show_data = TRUE)
```



NB: here, the marginal prediction is the average of the predicted values considering that any given flipper length could be found in either sex and in any species with probability reflecting the sampled data → breaks down correlations between predictors, but preserve the distributions of non-focal predictors (i.e. sex & species sampled more weigh more)

Interpreting coefficients — conditional and marginal predictions

Marginal predictions of the effect of `flipper_len` on `body_mass` ("population" method)

```
marg_pop_pred <- estimate_means(
  fit_flipper_3var,
  by = "flipper_len",
  estimate = "population")

as_tibble(marg_pop_pred)
```

```
# A tibble: 10 x 7
  flipper_len  Mean    SE CI_low CI_high     t    df
        <dbl> <dbl> <dbl>  <dbl>   <dbl> <dbl> <int>
1         172  3627.  84.0  3462.   3792.  43.2   328
2         179. 3758.  65.8  3629.   3888.  57.1   328
3         185. 3890.  47.9  3795.   3984.  81.1   328
4         192. 4021.  31.0  3960.   4082. 130.    328
5         198. 4152.  18.0  4117.   4187. 231.    328
# i 5 more rows
```

```
expand_grid(
  flipper_len = seq(172, 231, length = 10),
  sex = penguins_clean$sex,
  species = penguins_clean$species) -> grid_pop

grid_pop |>
  mutate(pred = predict(
    fit_flipper_3var,
    newdata = grid_pop)) -> grid_pop

grid_pop |>
  summarise(Mean = mean(pred), .by = "flipper_len") |>
  arrange(flipper_len)
```

```
# A tibble: 10 x 2
  flipper_len  Mean
        <dbl> <dbl>
1         172  3627.
2         179. 3758.
3         185. 3890.
4         192. 4021.
5         198. 4152.
# i 5 more rows
```

Generalised Linear Models (GLM)

GLM — why the need?

Classic linear models (LM) are useful to fit many phenomena, including many for which the relationship between a predictor and the response is not linear (e.g. polynomials), but they share the limitation that they are designed to approximate a data generating process that is intrinsically Gaussian (remember that the error is assumed normally distributed)

LM won't work well for:

- binary data (e.g. the presence/absence of a species)
- proportion (e.g. the hatching success in a nest), especially if values get close to 0 or 1
- count data (e.g. the number of species caught on camera), especially if counts are low
- time to event data (e.g. time to dispersion), especially if times are short

For these situations GLM are the first choice!

NB: logistic regression & poisson regressions are examples of GLM

GLM — notations of a GLM: simple notation

$$\eta_i = \beta_0 + \beta_1 \times x_{1,i} + \beta_2 \times x_{2,i} + \cdots + \beta_p \times x_{p,i}$$

with

- $\mathsf{E}(\mathsf{Y}) = \mu = g^{-1}(\eta)$
- $\mathsf{Var}(\mathsf{Y}) = \phi \mathsf{V}(\mu)$

We call:

- η the linear predictor
- g the link function (g^{-1} is the inverse link function)
- V the variance function
- ϕ is the dispersion parameter

NB:

- it is very much like an LM but there are new elements
- in GLM, if $\phi = 1$ (the usual case) the variance in the response is directly a function of the mean and thus of η, so no need for σ in this case!

GLM — data generating process: data simulation

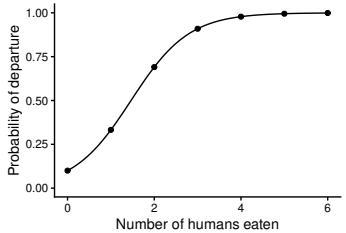
Let's reenter **God(dess) mode** → we pretend we know how her world generates data

GLM — data generating process: data simulation

Let's reenter **God(dess) mode** → we pretend we know how her world generates data

Emerald Lion informed us that in her world aliens depart to another planet following this rule:

$$P(\text{departure}_i) = \frac{1}{1 + \exp^{-(-2.197 + 1.5 \times \text{humans\_eaten}_i)}}$$



NB: the function $\frac{1}{1 + e^{-x}}$ is called the standard logistic function and is readily available in R via `plogis(x)`

GLM — data generating process: data simulation

Let's reenter **God(dess) mode** → we pretend we know how her world generates data

Emerald Lion informed us that in her world aliens depart to another planet following this rule:

$$P(\text{departure}_i) = \frac{1}{1 + \exp^{-(-2.197 + 1.5 \times \text{humans\_eaten}_i)}}$$

We can thus generate data on 35 aliens following E.L.'s recipe:

```
set.seed(123)
Alien2 <- tibble(humans_eaten = rep(0:6, each = 5),
                 prob_departure = plogis(-2.197 + 1.5*humans_eaten),
                 departed = rbinom(n = 35, size = 1, prob = prob_departure))
```

GLM — data generating process: data check

```
Alien2 |> slice(1:17) |> print(n = Inf)
```

```
# A tibble: 17 x 3
   humans_eaten prob_departure departed
          <int>          <dbl>    <int>
 1            0          0.100        0
 2            0          0.100        0
 3            0          0.100        0
 4            0          0.100        0
 5            0          0.100        1
 6            1          0.332        0
 7            1          0.332        0
 8            1          0.332        1
 9            1          0.332        0
10            1          0.332        0
11            2          0.691        0
12            2          0.691        1
13            2          0.691        1
14            2          0.691        1
15            2          0.691        1
16            3          0.909        1
17            3          0.909        1
```

```
Alien2 |> slice(18:35) |> print(n = Inf)
```

```
# A tibble: 18 x 3
   humans_eaten prob_departure departed
          <int>          <dbl>    <int>
 1            3          0.909        1
 2            3          0.909        1
 3            3          0.909        0
 4            4          0.978        1
 5            4          0.978        1
 6            4          0.978        1
 7            4          0.978        0
 8            4          0.978        1
 9            5          0.995        1
10            5          0.995        1
11            5          0.995        1
12            5          0.995        1
13            5          0.995        1
14            6          0.999        1
15            6          0.999        1
16            6          0.999        1
17            6          0.999        1
18            6          0.999        1
```

GLM — model fitting: frequentist approach

In practice, model parameters are unknown, so we have to estimate them
(God mode is again switched off)

```
fit_alien_glm <- glm(departed ~ humans_eaten, data = Alien2, family = binomial(link = "logit"))
```

NB:

- we used `glm()` rather than `lm()`
- we have a new term `binomial(link = "logit")` to indicate that the data are generated via a binomial distribution and to indicate which link the fit must use[2]

[2] the link is the function that converts the linear predictor into the scale of the response, or as here into a probability (more on that later)

GLM — model fitting: frequentist approach

The summary table is not very different from those produced with `lm()` either

```
summary(fit_alien_glm)
```

```
Call:
glm(formula = departed ~ humans_eaten, family = binomial(link = "logit"),
    data = Alien2)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)   -1.5952     0.8129  -1.962  0.04973 *
humans_eaten   1.0001     0.3411   2.932  0.00337 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 43.574  on 34  degrees of freedom
Residual deviance: 27.667  on 33  degrees of freedom
AIC: 31.667

Number of Fisher Scoring iterations: 5
```

GLM — model fitting: Bayesian approach with {brms}

Fitting GLM with {brms} is slightly different since it uses a specific family for binary data[3]

```
fit_alien_glm_brms <- brm(departed ~ humans_eaten, data = Alien2,
                          family = bernoulli(link = "logit"),
                          iter = 10000) # more needed than default this time
summary(fit_alien_glm_brms) # or fixef(fit_alien_lm_brms) for a focus on coefficients
```

```
 Family: bernoulli
  Links: mu = logit
Formula: departed ~ humans_eaten
   Data: Alien2 (Number of observations: 35)
  Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
         total post-warmup draws = 20000

Regression Coefficients:
             Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept       -1.81      0.87    -3.67    -0.28 1.00    11904    11479
humans_eaten     1.09      0.35     0.50     1.89 1.00     7029     8505

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

[3] the Bernoulli distribution is a special case of the binomial distribution when there is only 1 trial

GLM — model fitting: Bayesian approach with {nimble}

Defining the GLM by hand with {nimble} is relatively simple...

```
code_glm <- nimbleCode({
  for (i in seq_len(N)) {
      # model mimicking the data generating process
      logit(p[i]) <- intercept + slope*humans_eaten[i]
      departed[i] ~ dbin(p[i], 1)
  }
  # prior
  intercept ~ dnorm(mean = 0, sd = 2)
  slope ~ dnorm(mean = 0, sd = 2)
})

constants_glm <- list(N = nrow(Alien2),
                      humans_eaten = Alien2$humans_eaten)

inits_glm <- list(intercept = 0, slope = 0)
```

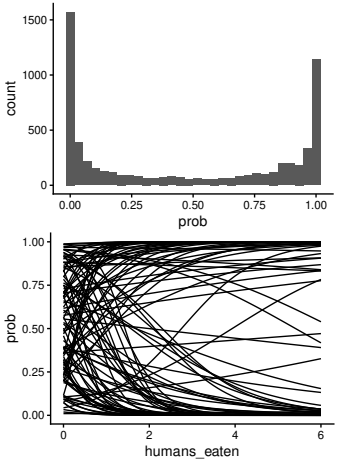
GLM — model fitting: Bayesian approach (prior predictive checking)

… but again we need to make sure the priors are sensible(here it would be even more difficult to foresee)

```
set.seed(123)
n_sims2 <- 100

prior_intercept2 <- rnorm(n = n_sims2, mean = 0, sd = 2)
prior_slope2 <- rnorm(n = n_sims2, mean = 0, sd = 2)

expand_grid(sim = seq_len(n_sims2),
            humans_eaten = seq(0, 6, by = 0.1)) |>
  mutate(prob = plogis(prior_intercept2[sim] +
      prior_slope2[sim] * humans_eaten)) -> prior_pred2

ggplot(prior_pred2) +
  geom_histogram(aes(x = prob))
```

```
ggplot(prior_pred2) +
  geom_line(aes(y = prob, x = humans_eaten, group = sim))
```

- it looks OK



GLM — model fitting: Bayesian approach with {nimble}

We can now fit the model with {nimble}

```
fit_alien_glm_nimb <- nimbleMCMC(code = code_glm,
                                 constants = constants_glm,
                                 data = list(departed = Alien2$departed),
                                 inits = inits_glm, nchains = 4, setSeed = 1:4,
                                 nburnin = 4000)
fit_alien_glm_nimb_d <- as_draws(fit_alien_glm_nimb)
summary(fit_alien_glm_nimb_d)
```

```
fit_alien_glm_nimb_d <- as_draws(fit_alien_glm_nimb)
summary(fit_alien_glm_nimb_d)
```

```
# A tibble: 2 x 10
  variable   mean median    sd   mad     q5    q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl> <dbl> <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
1 intercept -1.46 -1.42  0.738 0.724 -2.73  -0.318  1.00    1326.    2476.
2 slope      1.00  0.977 0.321 0.312  0.526  1.57   1.00    1256.    2159.
```

GLM — model fitting results: estimates compared

Let's compare the results

`glm()`

```
# A tibble: 2 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     -1.60        0.813  -3.43    -0.143
2 humans_eaten     1.00        0.341   0.440    1.82
```

`brms::brm()`

```
# A tibble: 2 x 5
  Parameter    Estimate Est.Error   Q2.5  Q97.5
  <chr>           <dbl>     <dbl>  <dbl>  <dbl>
1 Intercept       -1.81     0.866 -3.67  -0.281
2 humans_eaten     1.09     0.354  0.499  1.89
```

`nimble::nimbleMCMC()`

```
# A tibble: 2 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept -1.46 0.738 -3.06   -0.118
2 slope      1.00 0.321  0.440   1.71
```

GLM — model fitting results: estimates compared

Let's compare the results

`glm()`

```
# A tibble: 2 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     -1.60        0.813  -3.43    -0.143
2 humans_eaten     1.00        0.341   0.440    1.82
```

`brms::brm()`

```
# A tibble: 2 x 5
  Parameter    Estimate Est.Error   Q2.5  Q97.5
  <chr>           <dbl>     <dbl>  <dbl>  <dbl>
1 Intercept       -1.81     0.866 -3.67  -0.281
2 humans_eaten     1.09     0.354  0.499  1.89
```

`nimble::nimbleMCMC()`

```
# A tibble: 2 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept -1.46 0.738 -3.06   -0.118
2 slope      1.00 0.321  0.440   1.71
```

- the estimates are in agreement and all intervals include ground truth values

GLM — model fitting results: estimates compared

Let's compare the results

`glm()`

```
# A tibble: 2 x 5
  Parameter    Estimate `Std. Error` `2.5 %` `97.5 %`
  <chr>           <dbl>        <dbl>   <dbl>    <dbl>
1 (Intercept)     -1.60        0.813   -3.43   -0.143
2 humans_eaten     1.00        0.341    0.440   1.82
```

`brms::brm()`

```
# A tibble: 2 x 5
  Parameter    Estimate Est.Error   Q2.5  Q97.5
  <chr>           <dbl>     <dbl>  <dbl>  <dbl>
1 Intercept       -1.81     0.866 -3.67  -0.281
2 humans_eaten     1.09     0.354  0.499  1.89
```

`nimble::nimbleMCMC()`

```
# A tibble: 2 x 5
  variable   mean    sd `2.5%` `97.5%`
  <chr>     <dbl> <dbl>  <dbl>   <dbl>
1 intercept -1.46 0.738 -3.06   -0.118
2 slope      1.00 0.321  0.440   1.71
```

- the estimates are in agreement and all intervals include ground truth values
- we never obtained -2.197 (intercept) or 1.5 (slope) because, once again, a sample is unlikely to reflect perfectly the population-level values

GLM — model fitting: interlude on link functions

What is the so-called *logit* (or logistic unit, or log-odds)?

```
fit_alien_glm <- glm(departed ~ humans_eaten, data = Alien2, family = binomial(link = "logit"))
```

- it is the inverse of the standard logistic function, and is defined as: $\text{logit}(p) = \log \frac{p}{1-p}$

proof:

$$\frac{1}{1+e^{-\text{logit}(p)}} = \frac{1}{1+e^{-\log \frac{p}{1-p}}} = \frac{1}{1+\frac{1}{e^{\log \frac{p}{1-p}}}} = \frac{1}{1+\frac{1}{\frac{p}{1-p}}} = \frac{1}{1+\frac{1-p}{p}} = \frac{1}{\frac{p+1-p}{p}} = \frac{1}{\frac{1}{p}} = p$$

GLM — model fitting: interlude on link functions

What is the so-called *logit* (or logistic unit, or log-odds)?

```
fit_alien_glm <- glm(departed ~ humans_eaten, data = Alien2, family = binomial(link = "logit"))
```

- it is the inverse of the standard logistic function, and is defined as: $\text{logit}(p) = \log\frac{p}{1-p}$

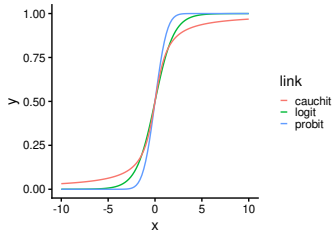
proof:

$$\frac{1}{1+e^{-\text{logit}(p)}} = \frac{1}{1+e^{-\log\frac{p}{1-p}}} = \frac{1}{1+\frac{1}{e^{\log\frac{p}{1-p}}}} = \frac{1}{1+\frac{1}{\frac{p}{1-p}}} = \frac{1}{1+\frac{1-p}{p}} = \frac{1}{\frac{p+1-p}{p}} = \frac{1}{\frac{1}{p}} = p$$

- it is the scale in which the linear predictor of the current GLM is expressed since the logit is allowed to vary between $-\infty$ and ∞ while p is constrained to vary between 0 and 1
- other link functions are possible for the binomial family (e.g. cauchit, probit), they are S-shaped too, but not identical

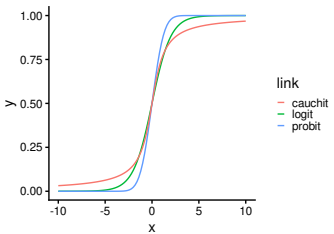
GLM — model fitting: interlude on link functions

Some link functions used for the binomial family



GLM — model fitting: interlude on link functions

Some link functions used for the binomial family



NB:

- the logit is a so-called *canonical link function*, the one with the best mathematical properties among alternatives, which is why it is the default of `binomial()`
- in Bayesian statistics there are fewer benefits to stick to canonical links compared to frequentist statistics, but the MCMC sampling can be difficult for some non-canonical links and how to interpret parameters can be even less straightforward than what we are about to see in a few slides → if you begin, stick to logit for binomial (& log for Poisson GLM)

GLM — Poisson GLM, data

Another very useful GLM is the Poisson GLM, which is used to fit count data

This time, we will directly use real data, the ones about the birds in Switzerland which you should already have downloaded:

```
territories <- read_csv("data_MHB/data-territories.csv")
species_count <- read_csv("data_MHB/header-species-count.csv")

territories |>
    summarise(territories_sum = sum(territories_total, na.rm = TRUE),
              .by = c(nameeg, namelt)) |>
    slice_max(territories_sum, n = 6) |>
    pull(namelt) -> top6sp

territories |>
    filter(namelt %in% top6sp & year == 2000) |>
    summarise(territories_top6 = sum(territories_total, na.rm = TRUE),
              .by = c(nameeg, samplearea_id)) |>
    left_join(species_count |> filter(year == 2000)) -> territories_top6_2000
```

GLM — Poisson GLM, data

Another very useful GLM is the Poisson GLM, which is used to fit count data

This time, we will directly use real data, the ones about the birds in Switzerland which you should already have downloaded:

```
territories_top6_2000
```

```
# A tibble: 1,584 x 12
  nameeg samplearea_id territories_top6 `x-koord` `y-koord`   lon   lat  year route_length
  <chr>          <dbl>            <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>
1 Coal ~           268                8   2725500   1182500  9.08  46.8  2000         4444
2 Coal ~           269                1   2497500   1118500  6.11  46.2  2000         6232
3 Coal ~           270               34   2725500   1214500  9.09  47.1  2000         4081
4 Coal ~           271                0   2503500   1134500  6.18  46.4  2000         4483
5 Coal ~           272               34   2725500   1230500  9.10  47.2  2000         4939
# i 1,579 more rows
# i 3 more variables: median_elevation <dbl>, n_visits <dbl>, n_recorded_species <dbl>
```

GLM — Poisson GLM, fit

Let's model the number of recorded number of territories occupied at a sample area (in 2000) according to the median elevation and the number of visits performed:

```
fit_bird_glm <- glm(territories_top6 ~ median_elevation + n_visits,
                    data = territories_top6_2000,
                    family = poisson(link = "log"))
summary(fit_bird_glm)
```

```
Call:
glm(formula = territories_top6 ~ median_elevation + n_visits,
    family = poisson(link = "log"), data = territories_top6_2000)

Coefficients:
                    Estimate  Std. Error z value Pr(>|z|)
(Intercept)      -3.93424937  0.19283817  -20.40   <2e-16 ***
median_elevation -0.00043192  0.00001432  -30.16   <2e-16 ***
n_visits          2.40357555  0.06316170   38.05   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 28575  on 1583  degrees of freedom
Residual deviance: 20463  on 1581  degrees of freedom
AIC: 25772

Number of Fisher Scoring iterations: 6
```

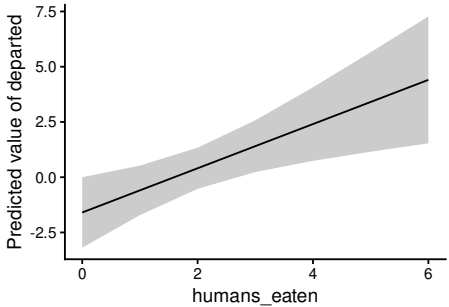
GLM — predictions and interpretation

At the scale of the linear predictor (here logit and then log), the interpretation is exactly as in LM

```
coef(fit_alien_glm)
```

```
(Intercept) humans_eaten
  -1.595208     1.000088
```

```
estimate_link(fit_alien_glm) |>
  plot(numeric_as_discrete = FALSE)
```



Interpretation

- on average, *the logit of departure* is -1.6 when no human is eaten
- on average, every additional human eaten increases *the logit of departure* by 1

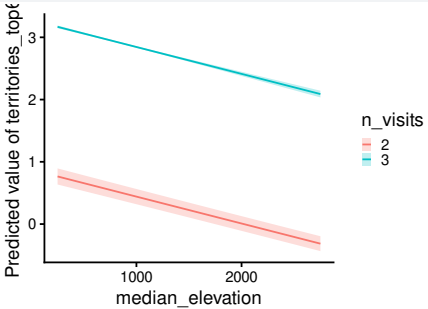
GLM — predictions and interpretation

At the scale of the linear predictor (here logit and then log), the interpretation is exactly as in LM

```
coef(fit_bird_glm)
```

```
      (Intercept) median_elevation         n_visits
-3.9342493729     -0.0004319178      2.4035755506
```

```
estimate_link(fit_bird_glm) |>
  plot()
```



Interpretation

- on average, *the log number of territories* is -3.93 when the median elevation is 0m and the number of visit is 0
- on average, every additional meter in median elevation increases *the log number of territories* by -0.000432
- on average, every additional visit increases *the log number of territories* by 2.4

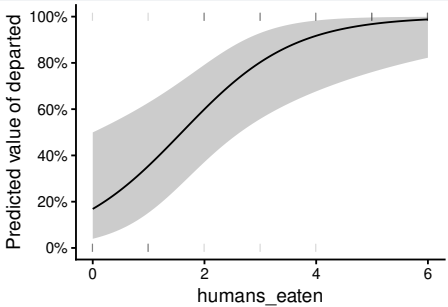
GLM — predictions and interpretation

At the scale of the response variable, predictions and interpretation differ

```
coef(fit_alien_glm)
```

```
(Intercept) humans_eaten
  -1.595208     1.000088
```

```
estimate_relation(fit_alien_glm,
  by = list(humans_eaten = seq(0, 6, length = 100))) |>
  plot(show_data = TRUE, numeric_as_discrete = FALSE)
```



Interpretation

- on average, every additional human eaten increases *the odds of departure* $e^{\text{slope}} = 2.72$ *fold*

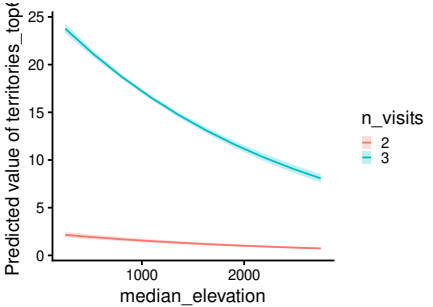
GLM — predictions and interpretation

At the scale of the response variable, predictions and interpretation differ

```
coef(fit_bird_glm)
```

```
      (Intercept) median_elevation         n_visits
-3.9342493729     -0.0004319178     2.4035755506
```

```
estimate_relation(fit_bird_glm) |>
  plot(show_data = FALSE) # no data points for clarity
```



Interpretation

- on average, every additional meter in median elevation increases the number of territories by $100 \times (e^{\text{slope}} - 1)$ = -0.0432%
- on average, every additional 1000 meter in median elevation increases the number of territories by $100 \times (e^{\text{slope} \times 1000} - 1) =$ -35.1%

Tests

Tests — main testing strategies

Context	Frequentist approach	Bayesian approach
compare non-nested (not a *proper* test)	AIC / BIC	BF / LOO-CV or WAIC comparison
compare two nested models (general)	F-test (for LM) / LRT (for GLM*)	BF / LOO-CV or WAIC comparison / PIP
special case: **model vs null model**	F-test (for LM) / LRT (for GLM*) of full vs intercept-only	BF / LOO-CV or WAIC comparison / PIP
special case: **overall effect of a variable**	F-test (for LM) / LRT (for GLM*) of full vs variable dropped	BF / LOO-CV or WAIC comparison / PIP
coefficient vs a fixed value (usually 0)	Wald t/z-test** (for LM/GLM*) on the coefficient	BF (Savage-Dickey) / credible interval / PIP

* GLM with quasi family (i.e. $\phi \neq 1$), should be dealt with as an LM
** Wald tests perform poorly for small samples or parameters near a boundary, and alternatives exist (e.g. parametric bootstrap)

Tests — main testing strategies

Context	Frequentist approach	Bayesian approach
compare non-nested (not a *proper* test)	AIC / BIC	BF / LOO-CV or WAIC comparison
compare two nested models (general)	F-test (for LM) / LRT (for GLM*)	BF / LOO-CV or WAIC comparison / PIP
special case: **model vs null model**	F-test (for LM) / LRT (for GLM*) of full vs intercept-only	BF / LOO-CV or WAIC comparison / PIP
special case: **overall effect of a variable**	F-test (for LM) / LRT (for GLM*) of full vs variable dropped	BF / LOO-CV or WAIC comparison / PIP
coefficient vs a fixed value (usually 0)	Wald t/z-test** (for LM/GLM*) on the coefficient	BF (Savage-Dickey) / credible interval / PIP

* GLM with quasi family (i.e. $\phi \neq 1$), should be dealt with as an LM
** Wald tests perform poorly for small samples or parameters near a boundary, and alternatives exist (e.g. parametric bootstrap)

Technical vocabulary:

- AIC = Akaike information criterion
- BIC = Bayesian information criterion
- BF = Bayes factor
- LOO-CV = leave one out cross validation
- WAIC = widely applicable information criterion = Watanabe–Akaike information criterion
- LRT = likelihood ratio test
- PIP = posterior inclusion probability

Tests — main testing strategies

Context	Frequentist approach	Bayesian approach
compare non-nested (not a *proper* test)	AIC / BIC	BF / LOO-CV or WAIC comparison
compare two nested models (general)	F-test (for LM) / LRT (for GLM*)	BF / LOO-CV or WAIC comparison / PIP
special case: **model vs null model**	F-test (for LM) / LRT (for GLM*) of full vs intercept-only	BF / LOO-CV or WAIC comparison / PIP
special case: **overall effect of a variable**	F-test (for LM) / LRT (for GLM*) of full vs variable dropped	BF / LOO-CV or WAIC comparison / PIP
coefficient vs a fixed value (usually 0)	Wald t/z-test** (for LM/GLM*) on the coefficient	BF (Savage-Dickey) / credible interval / PIP

* GLM with quasi family (i.e. $\phi \neq 1$), should be dealt with as an LM
** Wald tests perform poorly for small samples or parameters near a boundary, and alternatives exist (e.g. parametric bootstrap)

Technical vocabulary:

- AIC = Akaike information criterion
- BIC = Bayesian information criterion
- BF = Bayes factor
- LOO-CV = leave one out cross validation
- WAIC = widely applicable information criterion = Watanabe–Akaike information criterion
- LRT = likelihood ratio test
- PIP = posterior inclusion probability

NB:

- whatever you do, the response variable data must be strictly identical (beware of values dropped automatically due to NAs in predictors)
- to reduce false positives, perform tests in the following order: full vs null → full vs variable dropped → coefficient vs a fixed value

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

F-test testing H_0: the variance explained by the 2 LMs is the same

```
fit_alien_lm      <- lm(size ~ humans_eaten, data = Alien)
fit_alien_lm_null <- lm(size ~ 1, data = Alien)
anova(fit_alien_lm, fit_alien_lm_null)
```

```
Analysis of Variance Table

Model 1: size ~ humans_eaten
Model 2: size ~ 1
  Res.Df    RSS Df Sum of Sq      F   Pr(>F)
1     10 31.031
2     11 76.153 -1   -45.122 14.541 0.003411 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

F-test testing H_0: the variance explained by the 2 LMs is the same

```
fit_alien_lm      <- lm(size ~ humans_eaten, data = Alien)
fit_alien_lm_null <- lm(size ~ 1, data = Alien)
anova(fit_alien_lm, fit_alien_lm_null)
```

```
Analysis of Variance Table

Model 1: size ~ humans_eaten
Model 2: size ~ 1
  Res.Df    RSS Df Sum of Sq      F   Pr(>F)
1     10 31.031
2     11 76.153 -1   -45.122 14.541 0.003411 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation: we significantly reject the null model in favour of the first model

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Bayes factor comparing 2 LM using {brms} (simple but wrong)

```
fit_alien_lm_brms      <- brm(size ~ humans_eaten, data = Alien)
fit_alien_lm_brms_null <- brm(size ~ 1, data = Alien)
BF_lm <- bayes_factor(fit_alien_lm_brms, fit_alien_lm_brms_null)
BF_lm$bf
```

```
[1] 222.6148
```

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Bayes factor comparing 2 LM using {brms} (simple but wrong)

```
fit_alien_lm_brms      <- brm(size ~ humans_eaten, data = Alien)
fit_alien_lm_brms_null <- brm(size ~ 1, data = Alien)
BF_lm <- bayes_factor(fit_alien_lm_brms, fit_alien_lm_brms_null)
BF_lm$bf
```

```
[1] 222.6148
```

Interpretation: the observed data are 222.6 times more likely under our first LM than under the null model

BUT there is unfortunately a problem, unlike posteriors which should not be too sensitive to priors, the Bayes factor is, and the default priors in {brms} are vague and improper which makes them unsuitable for the task

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Bayes factor comparing 2 LM using {brms} (prior fix)

```
priors_brms2 <- c(prior(normal(0, 5), class = "b"),
                  prior(normal(50, 5), class = "Intercept"),
                  prior(uniform(0, 5), class = "sigma", ub = 5))

fit_alien_lm_brms2      <- brm(size ~ humans_eaten, data = Alien, prior = priors_brms2)
fit_alien_lm_brms_null2 <- brm(size ~ 1, data = Alien, prior = priors_brms2[2:3, ])
BF_lm2 <- bayes_factor(fit_alien_lm_brms2, fit_alien_lm_brms_null2)
BF_lm2$bf
```

```
[1] 10.94815
```

Tests — example of comparison of 2 nested models

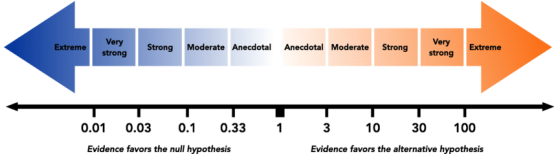
Is our model better than intercept-only model (i.e. null model)?

Bayes factor comparing 2 LM using {brms} (prior fix)

```
priors_brms2 <- c(prior(normal(0, 5), class = "b"),
                  prior(normal(50, 5), class = "Intercept"),
                  prior(uniform(0, 5), class = "sigma", ub = 5))

fit_alien_lm_brms2      <- brm(size ~ humans_eaten, data = Alien, prior = priors_brms2)
fit_alien_lm_brms_null2 <- brm(size ~ 1, data = Alien, prior = priors_brms2[2:3, ])
BF_lm2 <- bayes_factor(fit_alien_lm_brms2, fit_alien_lm_brms_null2)
BF_lm2$bf
```

```
[1] 10.94815
```

Interpretation: the observed data are 10.9 times more likely under our first LM than under the null model, which represents strong support in favour of our model



Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Posterior inclusion probability comparing 2 LM using {nimble}

```
code_pip <- nimbleCode({
  incl ~ dbin(p_incl, 1) # we add a switch
  for (i in seq_len(N)) {
    size[i] ~ dnorm(
  mean = intercept + incl*slope*humans_eaten[i], sd = sigma)
  }
  # priors
  intercept ~ dnorm(mean = 50, sd = 5)
  slope ~ dnorm(mean = 0, sd = 5)
  sigma ~ dunif(min = 0, max = 5)
})

constants_pip <- list(p_incl = 0.5, N = nrow(Alien), humans_eaten = Alien$humans_eaten)

inits_pip <- list(intercept = mean(Alien$size), slope = 0, sigma = 1)
```

NB:

- `p_incl` is fixed at 0.5, which reflects a neutral prior belief
- I centred `humans_eaten` to make the BF comparable to {brms} which internally use priors on parameters applied to centred variables
- if several predictors were present, `incl` would have to be multiplied to all corresponding regression coefficients

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Posterior inclusion probability comparing 2 LM using {nimble}

```
fit_alien_lm_nimb_pip <- nimbleMCMC(code = code_pip,
                                    constants = constants_pip,
                                    data = list(size = Alien$size),
                                    inits = inits_pip,
                                    nchains = 4, setSeed = 1:4,
                                    nburnin = 4000,
                                    progressBar = FALSE)

proba_incl <- mean(sapply(fit_alien_lm_nimb_pip, \(chain) mean(chain[, "incl"]))) # avg across chains
proba_incl
```

```
[1] 0.950375
```

Tests — example of comparison of 2 nested models

Is our model better than intercept-only model (i.e. null model)?

Posterior inclusion probability comparing 2 LM using {nimble}

```
fit_alien_lm_nimb_pip <- nimbleMCMC(code = code_pip,
                                    constants = constants_pip,
                                    data = list(size = Alien$size),
                                    inits = inits_pip,
                                    nchains = 4, setSeed = 1:4,
                                    nburnin = 4000,
                                    progressBar = FALSE)

proba_incl <- mean(sapply(fit_alien_lm_nimb_pip, \(chain) mean(chain[, "incl"]))) # avg across chains
proba_incl
```

```
[1] 0.950375
```

Interpretation

1. there is an 95% posterior probability that the effect of the number of humans eaten on the size of aliens is present, versus a 5% posterior probability that the data are equally consistent with the null model
2. since `p_incl` is fixed at 0.5, `proba_incl` directly determines BF which reduces to $\frac{\text{proba\_incl}}{(1 - \text{proba\_incl})} = 19.2$ (note, this is a bit different from {brms} BF because the latter internally centres the regressors which are multiplied to priors draws)

Tests — example of test on parameters

Are parameter estimates likely to be null?

t-tests testing H_0 that the parameter estimates are 0

```
summary(fit_alien_lm)$coefficients # or just summary(fit_alien_lm)
```

```
             Estimate Std. Error   t value     Pr(>|t|)
(Intercept)  48.835310  1.3454140 36.297608 5.987821e-12
humans_eaten  2.374922  0.6228057  3.813263 3.410945e-03
```

Interpretation: both parameters are significantly different from 0

NB:

- this is directly provided by the summary tables seen earlier

Tests — example of test on parameters

Are parameter estimates likely to be null?

using credibility intervals for testing H_0 that the parameter estimates are 0

```
fixef(fit_alien_lm_brms) # or just summary(fit_alien_lm_brms)
```

```
             Estimate Est.Error       Q2.5     Q97.5
Intercept    48.82913 1.5129858 45.8069388 51.849559
humans_eaten  2.37829 0.6995441  0.9932649  3.758505
```

Interpretation: 0 is not among the values considered plausible at the 95% credible level

NB:

- this is again provided by the summary tables seen earlier

Tests — conclusion

- testing parameters directly is much easier but it increases the risk of false positives when null hypothesis significance testing (NHST) is applied (this is why I recommended earlier the testing order going from overall to specific)

- in Bayesian statistics, the ordering isn't needed for formal error-rate control (unlike NHST), but the same order remains good scientific practice: conclusions about a coefficient are only meaningful if the broader model/variable is itself supported by the data

- in Bayesian statistics, the Bayes factor is appealing as it can potentially be applied to all testing situations, however it is very dependent on priors and thus on internal scaling of variables by software, which makes it a measure difficult to compare across studies, casting doubt on the usual "scale" of strength of evidence it is associated with

- in Bayesian statistics, under any continuous prior, a genuinely continuous slope is never exactly zero $\rightarrow H_0$ has no special status but is considered as just one hypothesis among many

Model construction

Model construction — TODO

- TODO

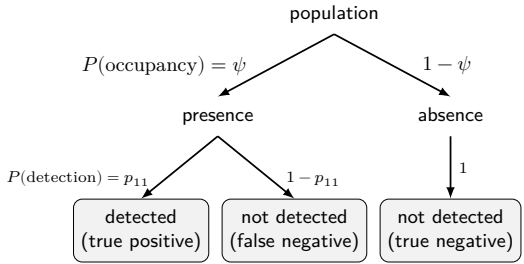
Assumptions

Assumptions — TODO

- TODO

Habitat occupancy model

Habitat occupancy model — recall the situation



We thus have a state space model (SSM):

- a non-observable layer with a state variable describing whether the habitat is occupied or not
- an observable outcome describing whether a particular species has been observed or not
- each of these levels can be modelled by a logistic GLM
- the 2 GLMs are not independent, it is a hierarchical model

Habitat occupancy model — fitting a no covariate model

We first simulate data and define the model

```
n_sites <- 100
n_visits <- 50

set.seed(123)
prob_occu <- 0.3; cond_prob_obs <- 0.1  # simulated truth

occu_truth <- rbinom(n = n_sites, size = 1, prob = prob_occu)

obs_mat <- matrix(NA, nrow = n_sites, ncol = n_visits)

for (site in seq_len(n_sites)) {
  obs_mat[site, ] <- occu_truth[site] * rbinom(n_visits, size = 1, prob = cond_prob_obs)
}

code_occu <- nimbleCode({
  for (site in 1:n_site) {
    z[site] ~ dbern(prob = psi) # occupancy status
    p[site] <- z[site] * p_cond  # detection prob
    for (visit in 1:n_visits) {
      y[site, visit] ~ dbern(p[site]) # observation outcome
    }
  }
  psi ~ dunif(0, 1); p_cond ~ dunif(0, 1)
})
```

Habitat occupancy model — fitting a no covariate model

We can now fit the model

```
constants_occu <- list(n_site = n_sites, n_visits = n_visits)

z_init <- as.numeric(rowSums(obs_mat) > 0) # initialise z: any detection -> known occupied
inits_occu <- list(p_cond = 0.5, psi = 0.1, z = z_init)

fit_occu <- nimbleMCMC(code = code_occu,
                       constants = constants_occu,
                       data = list(y = obs_mat),
                       inits = inits_occu,
                       nchains = 4, setSeed = 1:4,
                       niter = 10000, nburnin = 4000,
                       progressBar = FALSE)

summary(as_draws(fit_occu))
```

```
# A tibble: 2 x 10
  variable   mean median      sd     mad     q5    q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl>   <dbl>   <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
1 p_cond   0.0821 0.0820 0.00728 0.00728 0.0704 0.0942  1.00    5465.    5864.
2 psi      0.299  0.298  0.0459  0.0474  0.227  0.377   1.00    5211.    5583.
```

Habitat occupancy model — fitting a model with covariates

We first simulate data

```
n_sites <- 100
n_visits <- 50

set.seed(123)
prob_occu <- 0.3  # simulated truth
occu_truth <- rbinom(n = n_sites, size = 1, prob = prob_occu)

covar_detect_site <- rnorm(n = n_sites) # site-level covariate

# visit-level covariate, varies per site AND visit
covar_detect_visit <- matrix(rnorm(n = n_sites * n_visits), nrow = n_sites, ncol = n_visits)

# simulated truth for effect of covariates
beta0_true <- -2   # intercept
beta1_true <- 1    # site-level covariate slope
beta2_true <- 0.5  # visit-level covariate slope

p_true <- plogis(beta0_true +
                 beta1_true * covar_detect_site + # recycled across visits
                 beta2_true * covar_detect_visit) # site x visit matrix

obs_mat <- matrix(NA_real_, nrow = n_sites, ncol = n_visits)

for (site in seq_len(n_sites))
  obs_mat[site, ] <- occu_truth[site] * rbinom(n_visits, size = 1, prob = p_true[site, ])
```

Habitat occupancy model — fitting a model with covariates

We then define the model

```
code_occu <- nimbleCode({
  for (site in 1:n_site) {

    z[site] ~ dbern(prob = psi)

    for (visit in 1:n_visits) {

      logit(p_cond[site, visit]) <- beta0 +
        beta1 * covar_detect_site[site] +
        beta2 * covar_detect_visit[site, visit]

      p[site, visit] <- z[site] * p_cond[site, visit]

      y[site, visit] ~ dbern(p[site, visit])
    }
  }

  psi ~ dunif(0, 1)
  beta0 ~ dnorm(0, sd = 10)
  beta1 ~ dnorm(0, sd = 10)
  beta2 ~ dnorm(0, sd = 10)
})
```

Habitat occupancy model — fitting a model with covariates

We then set the constant and initialise the model

```
constants_occu <- list(n_site = n_sites,
                       n_visits = n_visits,
                       covar_detect_site = covar_detect_site,
                       covar_detect_visit = covar_detect_visit)

z_init <- as.numeric(rowSums(obs_mat) > 0) # initialise z: any detection -> known occupied

inits_occu <- function() {
  list(psi = runif(1, 0.1, 0.9),
       beta0 = rnorm(1, 0, 1),
       beta1 = rnorm(1, 0, 1),
       beta2 = rnorm(1, 0, 1),
       z = z_init)
}
```

Habitat occupancy model — fitting a model with covariates

Finally, we fit the model

```
fit_occu <- nimbleMCMC(code = code_occu,
                       constants = constants_occu,
                       data = list(y = obs_mat),
                       inits = inits_occu,
                       nchains = 4, setSeed = 1:4,
                       niter = 10000, nburnin = 4000,
                       progressBar = FALSE)

summary(as_draws(fit_occu))
```

```
# A tibble: 4 x 10
  variable   mean median     sd    mad     q5    q95  rhat ess_bulk ess_tail
  <chr>     <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl> <dbl>    <dbl>    <dbl>
1 beta0    -2.15  -2.15  0.102  0.101  -2.32  -1.98   1.00    2309.    3468.
2 beta1     1.04   1.04  0.0951 0.0956  0.886  1.20   1.00    2552.    4260.
3 beta2     0.586  0.584 0.0833 0.0831  0.451  0.724  1.00    3917.    5680.
4 psi       0.299  0.299 0.0460 0.0458  0.224  0.376  1.00    5068.    6133.
```

NB: the agreement between the true parameters and their estimates is very good!

Habitat occupancy model — towards more realistic models

Even our complex model made some particularly crude assumptions:

- we considered that the true probability of occupancy was fixed ($\psi = 0.3$ in our example) and thus did not vary in time or space
- we only look at one species in isolation

Relaxing these assumptions makes models quite more complicated, but fortunately, packages such as {nimbleEcology} or {unmarked} (a non-Bayesian alternative) have pre-built models to help you relax these assumptions with very little coding

Rahel will show you how to do that!

Conclusions

Conclusions

- modelling can be tricky, especially when relying on MCMC
- comparing the results from alternative packages relying on different set of assumptions can be useful to identify issues (default settings are not always best)
- simulating data is critical to make sure you do estimate what you think
- {nimble} is very flexible and can do much more than I showed you

Questions?

Developer corner

Ignore this slide — (technical info for Alex)

```
[1] "0.3 min of R running time"

R version 4.6.1 (2026-06-24)
Platform: x86_64-redhat-linux-gnu
Running under: Fedora Linux 44 (KDE Plasma Desktop Edition)

Matrix products: default
BLAS/LAPACK: FlexiBLAS OPENBLAS-OPENMP;  LAPACK version 3.12.1

attached base packages:
[1] stats     graphics  grDevices datasets  utils     methods   base

other attached packages:
 [1] modelbased_0.16.0 bayesplot_1.15.0  posterior_1.7.0   nimble_1.4.2      brms_2.23.0       Rcpp_1.1.2        lubridate_1.9.5
 [8] forcats_1.0.1     stringr_1.6.0     dplyr_1.2.1       purrr_1.2.2       readr_2.2.0       tidyr_1.3.2       tibble_3.3.1
[15] ggplot2_4.0.3     tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] Rdpack_2.6.6          gridExtra_2.3.1       inline_0.3.21         sandwich_3.1-3        rlang_1.3.0
 [6] magrittr_2.0.5        multcomp_1.4-31       otel_0.2.0            matrixStats_1.5.0     compiler_4.6.1
[11] loo_2.10.1            vctrs_0.7.3           reshape2_1.4.5        pkgconfig_2.0.3       crayon_1.5.3
[16] fastmap_1.2.0         backports_1.5.1       labeling_0.4.3        utf8_1.2.6            CoprManager_0.5.8
[21] rmarkdown_2.31        tzdb_0.5.0            pracma_2.4.6          nloptr_2.2.1          tinytex_0.60
[26] bit_4.6.0             xfun_0.60             jsonlite_2.0.0        parallel_4.6.1        R6_2.6.1
[31] stringi_1.8.9         RColorBrewer_1.1-3    StanHeaders_2.32.10   boot_1.3-32           numDeriv_2016.8-1.1
[36] estimability_2.0.0    rstan_2.32.7          knitr_1.51            zoo_1.9-0             parameters_0.29.2
[41] Matrix_1.7-6          splines_4.6.1         igraph_2.3.3          timechange_0.4.0      tidyselect_1.2.1
[46] rstudioapi_0.19.0     abind_1.4-8           yaml_2.3.12           codetools_0.2-20      curl_7.1.0
[51] pkgbuild_1.4.8        lattice_0.23-1        plyr_1.8.9            withr_3.0.3           bridgesampling_1.2-1
[56] bayestestR_0.18.1     S7_0.2.2              coda_0.19-4.1         evaluate_1.0.5        marginaleffects_0.32.0
[61] survival_3.8-9        RcppParallel_6.2.0    pillar_1.11.1         tensorA_0.36.2.1      checkmate_2.3.4
[66] stats4_4.6.1          reformulas_0.4.4      insight_1.5.2         distributional_0.8.1  generics_0.1.4
[71] vroom_1.7.1           hms_1.1.4             rstantools_2.7.0      scales_1.4.0          minqa_1.2.8
[76] xtable_1.8-8          glue_1.8.1            emmeans_2.0.4         tools_4.6.1           see_0.14.1
[81] data.table_1.18.4     lme4_2.0-6            mvtnorm_1.4-2         grid_4.6.1            rbibutils_2.4.1
[86] QuickJSR_1.10.0       datawizard_1.3.1      nlme_3.1-170          cli_3.6.6             Brobdingnag_1.2-9
[91] V8_8.2.0              gtable_0.3.6          digest_0.6.39         TH.data_1.1-5         farver_2.1.2
[96] htmltools_0.5.9       lifecycle_1.0.5       bit64_4.8.2           MASS_7.3-66
```